The Gizmo Guide

A Minimal Runtime for LLM Agents

Chris Ertel

Draft — February 2026

Gizmo is a minimal runtime for LLM agents modeled on process calculus and the BEAM. An agent is a process with a context stack, a mailbox, and three ops: `send`, `spawn`, `trap`. Everything else — tool use, memory, multi-agent coordination, human interaction — is built on top as mailbox-backed services.

Contents

1 Introduction 6
1.1 Key Ideas 7
1.2 Research Platform, Not Product 9
1.2.1 Here Be Dragons 9
1.2.2 Designed for Development with an AI Assistant 9
1.3 Quickstart 11
1.4 Hello, World! 12
2 Gizmo Anatomy 13
2.1 Overview of the Runtime 14
2.2 Overview of Execution State 15
2.2.1 Message-Driven Wakeups and Idle Restore 15
2.3 In-Depth on Interpolation 17
2.3.1 Interpolation in Context Stack Evaluation 17
2.3.2 Interpolation in Message Sending 19
2.4 In-Depth on Opcodes 20
2.4.1 `send` — Fire-and-Forget Message Delivery 20
2.4.2 `spawn` — Create a Child Process 20
2.4.3 `trap` — Interrupt Handler 21
2.5 In-Depth on Well-Known Services 23
2.5.1 `bash` — Shell Command Execution 23
2.5.2 `blackboard` — Key-Value Store 23
2.5.3 `human` — Terminal Output 23
2.5.4 `human_input` — Terminal Input 24
2.5.5 `exception` — Error Sink 24
2.5.6 `reaper` — Process Lifecycle 24
2.5.7 `watchdog` — Timer Service 24
2.5.8 Extending Well-Known Services 25
2.5.9 Services as Stateful Peers: A Document Pager 26
2.5.10 `batch` — Parallel Fan-Out 30
2.5.11 `eval` — Elixir Expression Evaluator 30
2.5.12 `factory` — Custom Service Factory 31
2.6 Worked Execution Cycle 33
2.6.1 Cycle 1 33
2.6.2 Cycle 2 34
2.6.3 Key Observations 34
3 Programming Gizmos 36
3.1 Key Techniques 37
3.1.1 Continuation Frames 37
3.1.2 Named Sections 37
3.1.3 The `@0` Loop 37

3.1.4 Self-Renewing Service Loops 38
3.1.5 Message-Conditional State Machines 39
3.2 Idioms 40
3.2.1 One-Shot Agent 40
3.2.2 Service Call (Message-Driven) 40
3.2.3 Interactive Loop (Setup + Sections) 40
3.2.4 Parent-Child with Trap 40
3.2.5 Disowned Peers with Blackboard Discovery 41
3.3 Debugging 42
3.3.1 Verbosity Levels 42
3.3.2 Dry Run 42
3.3.3 Tracing 42
3.3.4 Smoke Tests 43
3.4 Common Pitfalls 44
3.4.1 Pitfall 1: Using `${_msg}` for a response that hasn't arrived 44
3.4.2 Pitfall 2: Terse continuation frames 44
3.4.3 Pitfall 3: `@0` replaying one-time setup 44
3.4.4 Pitfall 4: Using `${_msg}` for work you just triggered 44
3.4.5 Pitfall 5: Autonomous loops with no wakeup source 44
3.4.6 Pitfall 6: Child referencing parent sections 44
3.4.7 Pitfall 7: Boot frame re-executing 45
3.4.8 Pitfall 8: Deferred transitions in message-driven mode 45
4 Future Work 46
4.1 Frame Tagging 47
4.1.1 Open Questions 47
4.2 Cognitohazard Vault 48
4.2.1 Open Questions 48
4.3 Pledge-for-Address and Pledge-for-Content 49
4.3.1 Open Questions 49
4.4 Session Persistence and Snapshotting 50
4.4.1 Open Questions 50
4.5 Debug Visualization 51
4.5.1 Open Questions 51
4.6 Self-Modifying Runtime (Blue-Green Gizmo) 52
4.6.1 Open Questions 52
5 Appendix A: In-Depth on Test Programs 53
5.1 01: Hello World 54
5.1.1 Listing 54
5.1.2 What to Learn 54
5.1.3 Questions for Reflection 54
5.1.4 Extension Projects 54
5.2 02: Bash 55
5.2.1 Listing 55
5.2.2 What to Learn 55

5.2.3 Questions for Reflection 55
5.2.4 Extension Projects 56
5.3 03: Blackboard 57
5.3.1 Listing 57
5.3.2 What to Learn 57
5.3.3 Questions for Reflection 58
5.3.4 Extension Projects 58
5.4 04: Fork (Spawn + Child Communication) 59
5.4.1 Listing 59
5.4.2 What to Learn 59
5.4.3 Questions for Reflection 60
5.4.4 Extension Projects 60
5.5 05: Echo Loop 61
5.5.1 Listing 61
5.5.2 What to Learn 61
5.5.3 Questions for Reflection 62
5.5.4 Extension Projects 62
5.6 06: Chat 63
5.6.1 Listing 63
5.6.2 What to Learn 63
5.6.3 Questions for Reflection 64
5.6.4 Extension Projects 64
5.7 07: Reaper 65
5.7.1 Listing 65
5.7.2 What to Learn 66
5.7.3 Questions for Reflection 66
5.7.4 Extension Projects 66
5.8 08: Lucky Number — Historical Removed Pattern 67
5.8.1 What to Learn 67
5.8.2 Questions for Reflection 67
5.8.3 Extension Projects 67
5.9 09: Lucky Number — Idle + Trap 68
5.9.1 Listing 68
5.9.2 What to Learn 69
5.9.3 Questions for Reflection 69
5.9.4 Extension Projects 70
5.10 10: Marketplace 71
5.10.1 What to Learn 71
5.10.2 Questions for Reflection 71
5.10.3 Extension Projects 71
5.11 11a: Named Spawn 72
5.11.1 Listing 72
5.11.2 What to Learn 72
5.11.3 Questions for Reflection 72

5.11.4 Extension Projects 72
5.12 11b: Each Hello 73
5.12.1 Listing 73
5.12.2 What to Learn 73
5.12.3 Questions for Reflection 73
5.12.4 Extension Projects 73
6 Appendix B: Dead Ends 74
6.1 Positional Args Stack (`$1`, `$2`, …) 75
6.2 Text-Based `<ops>`/`<frames>` Parsing 76
6.3 `spawn_link` for Children 77
6.4 `fork`/`join` as Primitives 78
6.5 Grinding Eval Loop as Default 79
6.6 Idle-by-Default 80
6.7 `untrap` as a Separate Op 81
6.8 Deferred Frame Transitions 82
6.9 Section-Based State Machines in Children 83
6.10 Checking `${_msg}` in Grind-Mode Children 84
7 Appendix C: How Gizmo Compares 85
7.1 vs. Stock LLMs (ChatGPT, Claude, etc.) 86
7.2 vs. OpenClaw 87
7.3 vs. ReAct 88
7.4 vs. LangGraph 89
7.5 vs. Erlang/OTP 90
7.6 vs. the π-Calculus 91
7.7 vs. Gas Town 92
7.8 vs. Ralph Loops 93
8 Appendix D: Developer Cheatsheet 94
8.1 Eval Cycle 95
8.2 Ops 96
8.2.1 Interpolation 96
8.2.2 Runtime Bindings 96
8.3 Well-Known Services 97
8.4 CLI Flags 98
8.4.1 Key Patterns 98
8.4.2 Pitfalls 98

1 Introduction

1.1 Key Ideas

Gizmo is built on a small number of core principles:

1. **Eval is the loop, not an operation.** The runtime calls the LLM in a loop. The LLM is the rewrite rule; the context stack is the string being rewritten. There is no "think" or "plan" step —every LLM call is an eval cycle that produces ops to execute and frames to continue with.
2. **Three ops only.** `send`, `spawn`, `trap`. There are no special-cased tool calling, memory, or orchestration primitives. Every capability is built on top of message passing.
3. **Everything is a mailbox.** A shell, a key-value store, a human, another agent—all are addressed the same way. An agent sends a message to a mailbox ID and doesn't know or care what's behind it.
4. **The context stack is the prompt.** Frames (strings) are concatenated bottom-up and sent to the LLM as the system prompt. The boot frame is always the first frame, enabling prompt caching. The LLM returns replacement frames each cycle, so the context stack is self-modifying.
5. **Interpolation before ops.** Bindings like `${name}` in the ops and frames returned by the LLM are resolved *before* ops execute. If an op schedules future work or causes a service to reply, the resulting message is observed on the *next* wakeup as `${_msg}`, not in the current cycle's own ops.

These ideas combine to produce a system where the LLM is the program counter: it reads the current context (frames), decides what to do (ops), and rewrites its own future instructions (replacement frames).

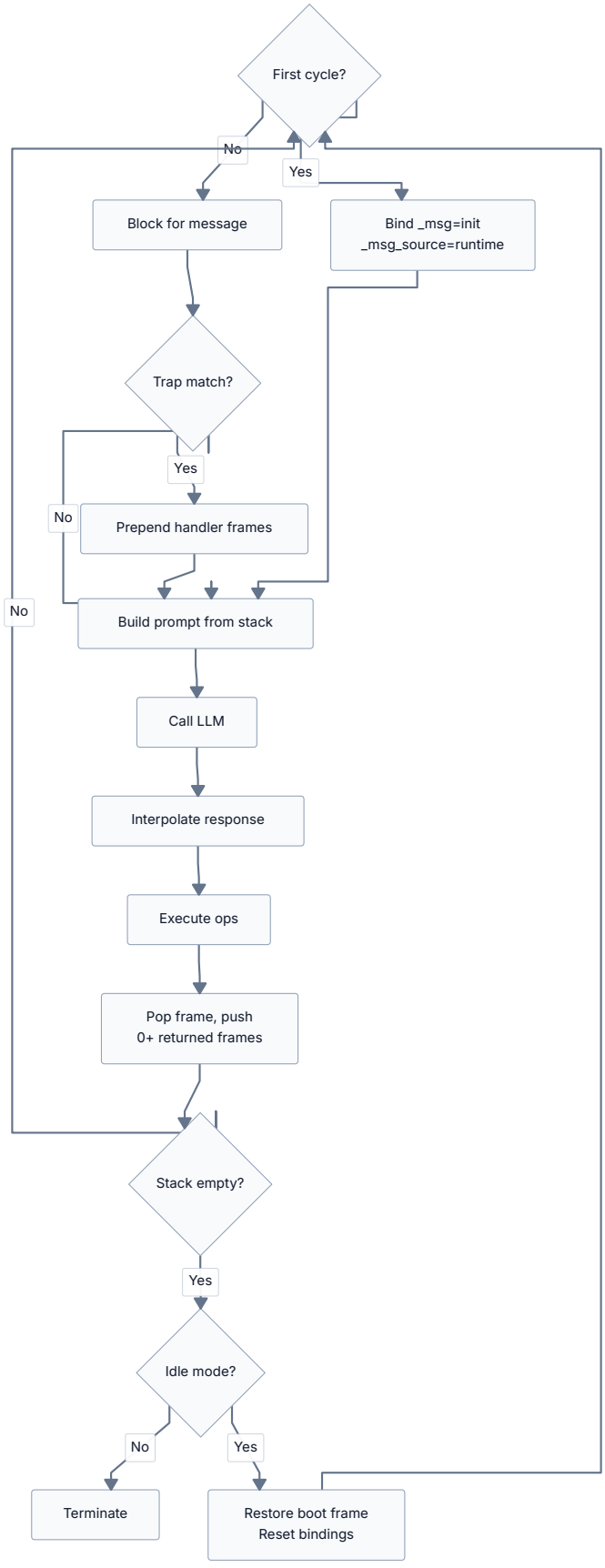

Figure 1: The eval cycle

1.2 Research Platform, Not Product

Gizmo is a research project. It is not a production framework, not a library, and not something you should deploy to serve real users. Understanding what this means—and why—is important before you invest time in it.

1.2.1 Here Be Dragons

Gizmo gives LLM agents direct access to shell commands, message passing, and process spawning. There is no sandbox, no permission system, and no content filtering. An agent that sends `rm -rf /` to the `bash` mailbox will execute exactly that.

This is by design. The runtime is a research tool for exploring what LLM agents can do when given minimal but composable primitives. Adding safety rails would obscure the behavior being studied. But it means:

- **Do not run untrusted boot frames.** A boot frame is arbitrary instructions to an LLM with shell access.
- **Do not expose Gizmo to the internet.** There is no authentication, no rate limiting, no input sanitization.
- **Monitor your API spend.** An agent that self-messages aggressively or schedules dense watchdog ticks can still burn through cycles quickly.
- **Read the boot frame before running it.** Understand what the agent will do before you let it do it.

Future work on pledge-for-address and pledge-for-content (Section 4.3) may eventually constrain agent capabilities, but today, agents have full access to everything the runtime provides.

1.2.2 Designed for Development with an AI Assistant

The entire runtime is a single Elixir script (`gizmo.exs`, roughly 6000 lines). This is unusual and intentional.

Why a single file?

- It can be pasted in full into an LLM context window. An AI assistant can read the entire runtime, understand it, and modify it—no multi-file navigation, no implicit dependencies, no build system.
- It eliminates the overhead of module boundaries, file layout decisions, and import management. When the goal is rapid iteration on runtime semantics, these are pure friction.
- It is trivially shareable. `scp gizmo.exs` gives someone the complete runtime.

What this means in practice:

- The code is organized as 18+ Elixir modules within the single file, with clear section boundaries.
- There are no external dependencies beyond `Req` (an HTTP client), fetched automatically via `Mix.install`.

- Documentation files (`ARCHITECTURE.md`, `PROMPTING.md`, etc.) are kept alongside the script and are written to be readable by both humans and LLMs.
- Development is done with heavy AI assistance. Prompts, code, and documentation are co-evolved.

If this approach bothers you, Gizmo is probably not for you—and that's fine.

1.3 Quickstart

Prerequisites:

- Elixir 1.19+ / Erlang/OTP 28+
- An Anthropic API key (or an OpenAI-compatible endpoint)

Installation:

```
git clone git@github.com:crertel/gizmo.git
cd gizmo
```

If you have Nix, the repo includes a `flake.nix`:

```
nix develop
```

Otherwise, install Elixir 1.19+ directly. Dependencies are fetched automatically on first run.

Set your API key:

```
export ANTHROPIC_API_KEY="sk-ant-..."
```

Generate a starter boot frame and run it:

```
elixir gizmo.exs --init my_task.txt
# Edit the task section in my_task.txt, then:
elixir gizmo.exs my_task.txt
```

Useful flags for getting started:

Flag	Effect
`-v`	Verbose mode—shows lifecycle events, ops, frames each cycle
`-vvv`	Maximum verbosity—includes bindings and full frame content
`--thinking`	Enable extended thinking (Anthropic only)
`--dry-run`	Print the full initial prompt and exit (no LLM call)
`--max-cycles N`	Limit eval cycles (default: 50, 0 = unlimited)
`--test`	Run built-in smoke tests

1.4 Hello, World!

The simplest possible Gizmo agent. Create a file `hello.txt`:

```
You are a one-shot greeter. Send a short, friendly hello message to the
'human' mailbox, then terminate by returning an empty frames array.
```

Run it:

```
elixir gizmo.exs hello.txt
```

The runtime loads the boot frame, calls the LLM, and the LLM returns something like:

```
{
  "ops": [{"op": "send", "mailbox": "human", "msg": {"text": "Hello there!"}}],
  "frames": [],
  "notes": {}
}
```

The runtime executes the `send` op (printing "Hello there!" to the terminal), sees that the returned frames are empty, and the agent terminates. One cycle, one message, done.

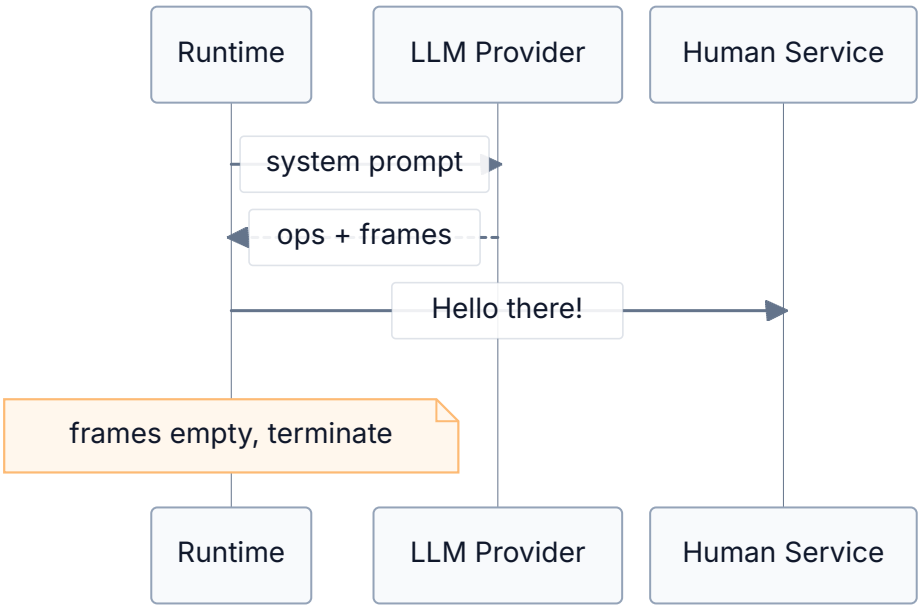

Figure 2: Hello world: one cycle, one message

This illustrates the fundamental contract: the LLM reads the context stack (the boot frame), decides what to do (send a greeting), specifies what comes next (nothing—empty frames), and the runtime handles the rest.

2 Gizmo Anatomy

2.1 Overview of the Runtime

The Gizmo runtime is an Elixir application organized as a supervision tree:

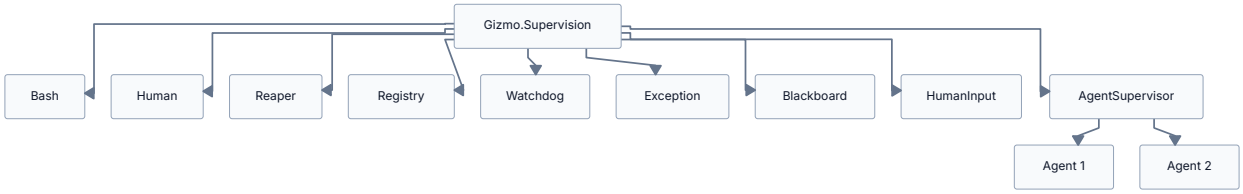

Figure 3: Supervision tree

The **Mailbox Registry** is an Elixir `Registry` in unique-key mode. Every service and every agent registers a mailbox ID that maps to its Erlang process PID. The `send` op resolves a mailbox ID through this registry to deliver messages.

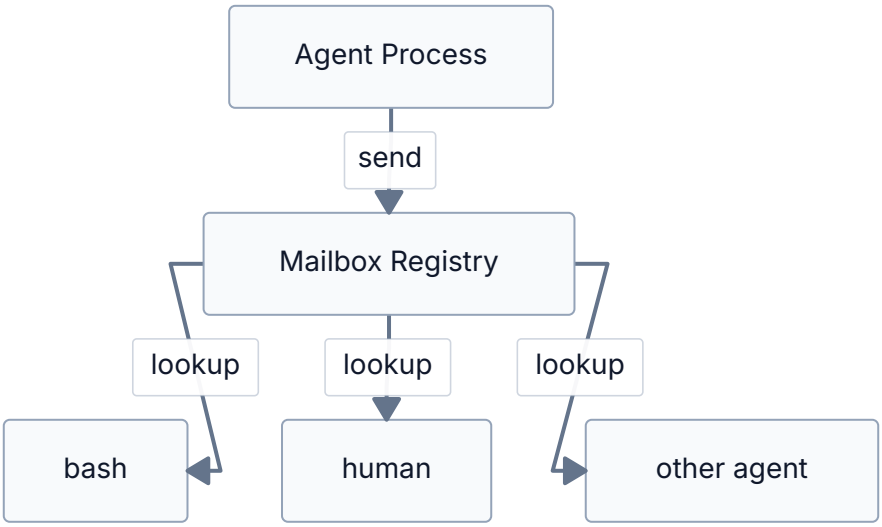

Figure 4: Message routing: `send` resolves mailbox ID to Erlang PID via the Registry

Well-known services are supervised GenServers with fixed mailbox names. They are started by the runtime at boot—not by agents. If a service crashes, the supervisor restarts it automatically. Services are independent of each other.

Agents run under a `DynamicSupervisor` with `:temporary` restart strategy—if an agent crashes, it is *not* restarted. Restarting from the boot frame would duplicate work and confuse parent processes waiting for child messages. Instead, a `Process.monitor` watcher sends a `child_died:<mailbox_id>` notification to the parent's mailbox.

2.2 Overview of Execution State

Each agent process carries the following state through its eval loop:

Field	Description
`context_stack`	List of frame strings. Concatenated bottom-up as the system prompt.
`mailbox_id`	This agent's address for receiving messages (e.g., `"agent_1"`).
`parent_id`	The spawning agent's mailbox ID, or `nil` for root/disowned agents.
`bindings`	Map of named values from `spawn` and the runtime (`_self`, `_parent`, `_msg`, `_msg_source`, `_payload`).
`trap`	A `{regex, handler_frames}` tuple, or `nil`. Single-slot interrupt handler.
`idle`	Boolean. If `true`, restores boot frame (instead of terminating) when frames drain to `[]`.
`messages_queue`	A per-agent `MessagesQueue` GenServer. Currently unused in the runtime path (retained for test assertions).

The **runtime bindings** are always available:

- `${_self}` — this agent's mailbox ID. Always present.
- `${_parent}` — the spawning agent's mailbox ID. Only for non-root, non-disowned children.
- `${_msg}` — text summary of the wake message (`"text"` field if present, else the full JSON). `"init"` on cycle 0.
- `${_msg_source}` — the sender's mailbox ID. `"runtime"` on cycle 0.
- `${_payload}` — full JSON of the wake message. `"{}"` on cycle 0.

2.2.1 Message-Driven Wakeups and Idle Restore

Current Gizmo has a single pacing model: **message-driven wakeups**. Every cycle after cycle 0 begins by blocking for a mailbox message. The message that woke the cycle is bound as `${_msg}`, `${_msg_source}`, and `${_payload}`.

`idle` controls only frame-exhaust behavior:

Mode	Behavior
Default	Agent wakes on a message, does work, and terminates when frames drain to `[]`.
`idle: true` / `--idle`	Agent wakes on a message, does work, then restores the boot frame instead of terminating when frames drain to `[]`. Bindings reset to `{_self, _parent}` plus fresh wake bindings.

Key invariants:

- **`_msg` binding:** Updated from the mailbox on every cycle after cycle 0.
- **Bindings from `spawn`:** Persist until the stack drains. On idle restore, only identity bindings and the new wake message remain.
- **Trap:** Fires between cycles, before prompt construction, when the wake message matches the regex.
- **Autonomous continuation:** If an agent needs to wake itself later, it must schedule a watchdog tick or send a message to `${_self}`.

2.3 In-Depth on Interpolation

Interpolation is the mechanism by which the runtime resolves references in the ops and frames returned by the LLM. Understanding when and how it runs is essential—most common mistakes stem from wrong assumptions about interpolation timing.

2.3.1 Interpolation in Context Stack Evaluation

When the LLM returns replacement frames, the runtime interpolates them *before* they become the new context stack. There are four interpolation mechanisms, resolved in this order:

1. **`@@` → escape sentinel.** Double-at becomes a sentinel that is restored to a literal `@` at the end. This prevents `@@end` markers from being confused with `@end` references.
2. **`$$` → escape sentinel.** Double-dollar becomes a sentinel restored to literal `$` at the end.
3. **`@name` / `@N` → section or frame injection.** `@N` injects the full text of frame N (0-indexed) from the *current* context stack. `@name` injects the contents of a named section (`@@name` … `@@end`). Injected text is **quoted verbatim**—any `$` in the injected content is escaped, so `${var}` inside a section stays literal.
4. **`${name}` → named binding resolution.** Resolved from the bindings map. If no binding with that name exists, the reference is left as the literal string `${name}`.
5. **Sentinels restored.** Escape sentinels become literal `@` and `$`.

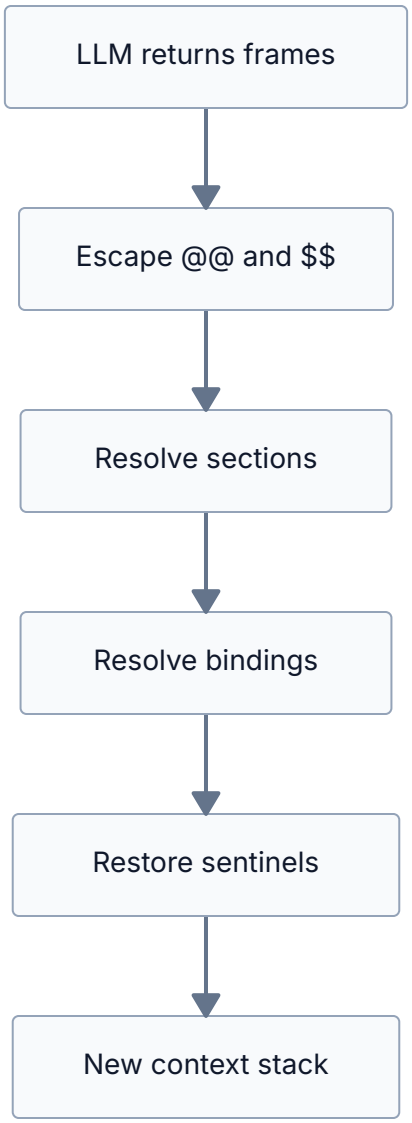

Figure 5: Interpolation resolution order

The critical consequence: **section content is injected before `${}` resolution, but the injected content's `$` characters are escaped.** So a section containing `${_msg}` will inject the literal string `${_msg}` into the frame—it will *not* resolve against the current bindings. This is the "quoted verbatim" guarantee.

Named sections are defined in frame text:

```
@@section-name
content goes here
multiple lines are fine
@@end
```

Rules:

- `@@section-name` must appear at the start of a line, followed by a newline.

- `@@end` must appear at the start of a line.
- Sections are scanned across all frames in the context stack. First match wins.
- Section markers are *left in* the system prompt—the LLM can see and reason about them.
- Sections persist across cycles via a cache. New definitions merge in; redefinitions overwrite.
- The regex is non-greedy: `@@name...@@end` matches the *first* `@@end`. Nested sections are not supported—keep sections flat.

2.3.2 Interpolation in Message Sending

When a `send` op executes, the `msg` field is interpolated against the current bindings map. This is the *same* interpolation pass that was already applied to the op when the LLM's response was processed.

The key timing: interpolation of the entire LLM response (ops *and* frames) happens in one pass, *before* any ops execute. So:

```
{
  "ops": [
    {"op": "send", "mailbox": "bash", "msg": {"command": "echo hello"}},
    {"op": "send", "mailbox": "human", "msg": {"text": "Got: ${_msg}"}}
  ]
}
```

In the second op, `${_msg}` still refers to the message that woke *this* cycle, not the future bash response. This is the most common source of confusion.

> **Rule of thumb**
> If you need data from a service response, you cannot use it in the same cycle's ops. Return a continuation frame and use the data on the next cycle, where it will arrive as `${_msg}`.

2.4 In-Depth on Opcodes

Gizmo has exactly three opcodes. Every agent capability is built from these primitives.

2.4.1 `send` — Fire-and-Forget Message Delivery

```
{"op": "send", "mailbox": "<target>", "msg": {"text": "<content>"}}
```

Delivers `msg` to the mailbox identified by `mailbox`. Non-blocking—the sender does not wait for a response or acknowledgment. The message is routed through the Mailbox Registry; if the target mailbox doesn't exist, the message is silently dropped.

The `msg` field must be a JSON object. It is interpolated (recursively, including nested values) before delivery, so `${_self}` in the message becomes the sender's mailbox ID.

2.4.2 `spawn` — Create a Child Process

```
{
  "op": "spawn",
  "frames": ["<child frame 1>", ...],
  "dest": "<binding_name>",
  "idle": true,
  "disown": true,
  "name": "worker",
  "model": "claude-..."
}
```

Creates a new agent process with the given frames as its context stack. The child's mailbox ID is stored in the parent's bindings under `dest`.

Frame content in `frames` is interpolated in the *parent's* context before being passed to the child. So `["@worker"]` resolves the `@@worker` section from the parent's frames. The child does *not* inherit the parent's section definitions—it gets plain text.

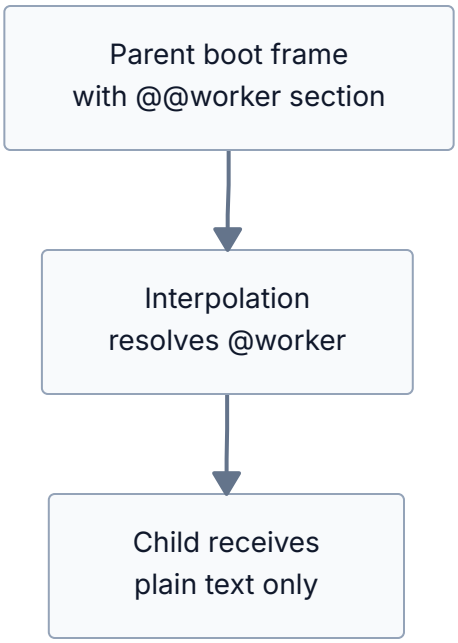

Figure 6: Spawn resolves sections in the parent's context

Optional fields:

Field	Effect
`idle`	Child restores boot frame on empty frames instead of terminating.
`disown`	No `${_parent}` binding, no death monitor. Fully independent peer.
`name`	Custom mailbox ID instead of auto-generated `agent_N`. Must be unique.
`model`	Override the LLM model for the child.

A `Process.monitor` watches non-disowned children. If a child crashes, a `child_died:<mailbox_id>` message is sent to the parent's mailbox.

2.4.3 `trap` — Interrupt Handler

```
{"op": "trap", "pattern": "<PCRE regex>", "frames": ["<handler frame>", ...]}
```

Registers a single-slot interrupt handler. Between eval cycles, when a message arrives that matches the PCRE regex `pattern`, the handler frames are *prepended* to the context stack and the cycle proceeds immediately. The bindings `${_interrupt}` and `${_interrupt_source}` are set.

Only one trap can be active. A new `trap` replaces the old one. Clear the trap with empty frames:

```
{"op": "trap", "pattern": ".*", "frames": []}
```

The trap persists across cycles. This is useful for handling child death notifications: register a trap for `"^child_died:"` once, and it fires whenever a child dies.

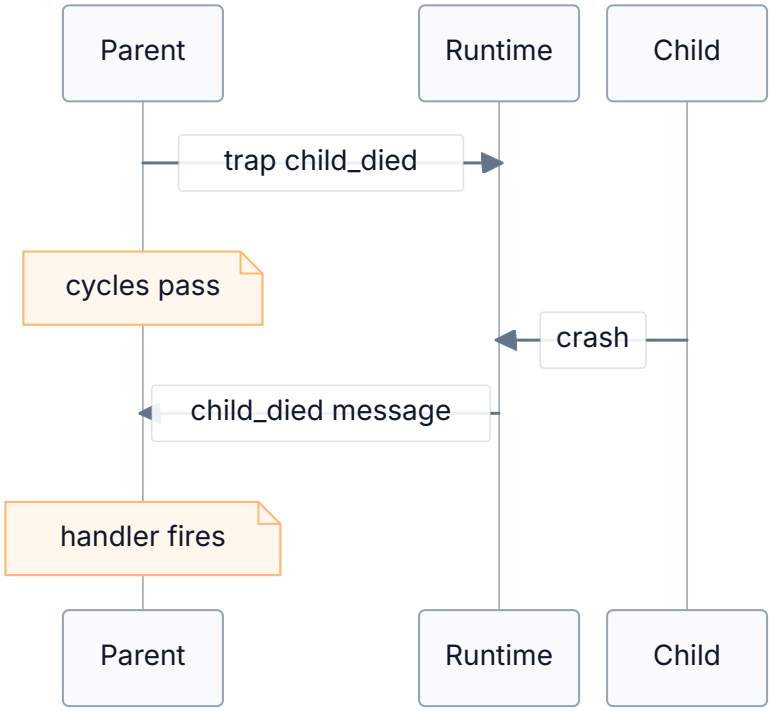

Figure 7: Trap fires when an inter-cycle message matches the pattern

2.5 In-Depth on Well-Known Services

The well-known services are supervised GenServers with fixed mailbox names. They are ordinary Erlang processes—not syscalls, not special runtime hooks. They communicate via the same `send`/`receive` protocol as agents.

2.5.1 `bash` — Shell Command Execution

Mailbox: `"bash"`

Send a JSON object with a `"command"` field; receive the output as `${_msg}` on the next cycle.

```
{"op": "send", "mailbox": "bash", "msg": {"command": "uname -a"}}
```

Output arrives as `${_msg}`. On failure: `"error: exit code N: ..."`.

With timeout and mode — add optional `"timeout"`, `"mode"`, and `"note"` fields:

```
{"op": "send", "mailbox": "bash", "msg": {"command": "find / -name '*.log'",
"timeout": 5000, "mode": "kill"}}
```

Mode	Behavior on timeout
`"kill"`	Command terminated. Agent receives `"error: timeout after Nms"`.
`"notify"`	Command keeps running. Agent receives a timeout notification with a handle. Can then send `{"action": "kill", "handle": "<h>"}` or `{"action": "wait", "handle": "<h>"}` to control the job.

Default timeout is set by `--bash-timeout` (default: 60000ms). Commands without a `"timeout"` field use the default timeout in kill mode.

2.5.2 `blackboard` — Key-Value Store

Mailbox: `"blackboard"`

Send JSON objects with an `"action"` field; result arrives as `${_msg}`:

- `{"action": "write", "key": "<key>", "value": "<value>"}` — returns `"ok"`.
- `{"action": "read", "key": "<key>"}` — returns the value (or empty string if key doesn't exist).

2.5.3 `human` — Terminal Output

Mailbox: `"human"`

Send a JSON object with a `"text"` field to display it on the user's terminal. Fire-and-forget—no response.

```
{"op": "send", "mailbox": "human", "msg": {"text": "Hello, user!"}}
```

2.5.4 `human_input` — Terminal Input

Mailbox: `"human_input"`

Send a JSON object with a `"prompt"` field. The user's typed line (trimmed) arrives as `${_msg}` on the next cycle.

```
{"op": "send", "mailbox": "human_input", "msg": {"prompt": "Enter your name: "}}
```

> **Tip**
> When combining output and an input prompt, send both in a single message to `human_input` separated by a newline. Sending to `human` and `human_input` as separate ops can produce garbled output because the two GenServers process messages independently.

2.5.5 `exception` — Error Sink

Mailbox: `"exception"`

Receives error reports from the runtime (agent retry exhaustion, cycle limit exceeded). Not typically addressed by agents directly.

2.5.6 `reaper` — Process Lifecycle

Mailbox: `"reaper"`

Send a JSON object with a `"target"` field containing the mailbox ID to kill. The reaper walks the parent chain from the target to verify the caller is an ancestor. Only ancestors can kill descendants —peer-to-peer kills are rejected.

```
{"op": "send", "mailbox": "reaper", "msg": {"target": "${child}"}}
```

The killed agent's parent receives a `child_died:` notification through the normal death-monitoring mechanism.

2.5.7 `watchdog` — Timer Service

Mailbox: `"watchdog"`

Send JSON objects with an `"action"` field. Ticks arrive as `"tick"` from source `"watchdog"`.

Message	Behavior
`{"action": "every", "ms": N}`	Periodic ticks every N milliseconds.
`{"action": "after", "ms": N}`	Single tick after N milliseconds.
`{"action": "cancel"}`	Cancel all timers for the sender.
`{"action": "list"}`	List active timers (reply sent back to sender).

Multiple timers stack. An agent can have several `every` and `after` timers simultaneously.

2.5.8 Extending Well-Known Services

Because services are ordinary GenServer processes registered in the Mailbox Registry, you can add new ones by following the same pattern:

1. Write a GenServer that implements `handle_cast({:mailbox_msg, content, source}, state)`.
2. Register it in the Mailbox Registry under a fixed name.
3. Start it under `Gizmo.Supervision` (add it to the children list).

The agent doesn't need to know that your new service exists at the language level—it just needs to `send` to the right mailbox name. You *do* need to document the service's protocol in the runtime preamble (or in a custom `--runtime` file) so the LLM knows what messages to send and what responses to expect.

For example, a hypothetical `timer` service that tracks elapsed time:

```
defmodule Gizmo.Services.Timer do
  use GenServer

  def start_link(_), do: GenServer.start_link(__MODULE__, %{})

  def init(state) do
    Gizmo.Mailbox.register("timer", nil)
    {:ok, state}
  end

  def handle_info({:mailbox_msg, _mailbox_id, {sender_mb, %{"action" =>
"start"}}}, state) do
    {:noreply, Map.put(state, sender_mb, System.monotonic_time(:millisecond))}
  end

  def handle_info({:mailbox_msg, _mailbox_id, {sender_mb, %{"action" =>
"elapsed"}}}, state) do
    start = Map.get(state, sender_mb, System.monotonic_time(:millisecond))
    elapsed = System.monotonic_time(:millisecond) - start
    Gizmo.Mailbox.route(sender_mb, {"timer",
      %{"text" => "#{elapsed}ms", "elapsed_ms" => elapsed}})
    {:noreply, state}
```

```
  end
end
```

The protocol is simple: send `{"action": "start"}` to begin, send `{"action": "elapsed"}` to get the duration. The agent addresses it like any other mailbox.

2.5.9 Services as Stateful Peers: A Document Pager

The timer example is simple, but it understates the power of the mailbox abstraction. Consider a more substantial service: a document pager that lets an agent read through a large file page by page, despite the agent's limited context window.

The design uses two modules: a **factory** (singleton service) and a **session** (one process per open document). When an agent sends `{"action": "open", "path": "/path"}` to the `"pager"` mailbox, the factory spawns a new session process, registers it with a unique mailbox ID, and tells the agent the ID. The agent then talks directly to the session. This means an agent can have multiple documents open simultaneously—each is a separate process with its own mailbox.

```
defmodule Gizmo.Services.Pager do
  @moduledoc "Factory: receives 'open' requests, spawns session processes."
  use GenServer

  def start_link(mailbox_id \\ "pager") do
    GenServer.start_link(__MODULE__, mailbox_id)
  end

  def init(mailbox_id) do
    Gizmo.Mailbox.register(mailbox_id)
    {:ok, %{mailbox_id: mailbox_id, counter: 0}}
  end

  def handle_info({:mailbox_msg, _mailbox_id, {sender_mb, %{"action" => "open",
"path" => path}}}, state) do
    path = String.trim(path)

    case File.read(path) do
      {:ok, content} ->
        lines = String.split(content, "\n")
        id = "pager_#{state.counter}"
        {:ok, _} = Gizmo.Services.PagerSession.start(id, lines, sender_mb)
        line_count = length(lines)
        Gizmo.Mailbox.route(sender_mb, {state.mailbox_id,
          %{"text" => "opened:#{id}:#{line_count} lines", "session" => id, "lines"
=> line_count}})
        {:noreply, %{state | counter: state.counter + 1}}

      {:error, reason} ->
```

```
        Gizmo.Mailbox.route(sender_mb, {state.mailbox_id,
          %{"text" => "error:#{reason}", "error" => to_string(reason)}})
        {:noreply, state}
    end
  end
end

defmodule Gizmo.Services.PagerSession do
  @moduledoc "Per-document session: holds file contents, cursor, page size."
  use GenServer

  @default_page_size 40

  def start(id, lines, owner_mailbox_id) do
    GenServer.start(__MODULE__, {id, lines, owner_mailbox_id})
  end

  def init({id, lines, owner_mb}) do
    Gizmo.Mailbox.register(id)
    monitor_ref =
      case Gizmo.Mailbox.lookup(owner_mb) do
        {:ok, pid} -> Process.monitor(pid)
        {:error, _} -> nil
      end
    {:ok, %{id: id, lines: lines, total: length(lines),
            cursor: 0, page_size: @default_page_size,
            owner_mb: owner_mb, monitor_ref: monitor_ref}}
  end

  def handle_info({:mailbox_msg, _id, {sender_mb, %{"action" => "next"}}}, state)
do
    {page_text, new_cursor} = get_page(state.lines, state.cursor, state.page_size,
state.total)
    from = state.cursor + 1
    to = min(state.cursor + state.page_size, state.total)
    header = "lines #{from}-#{to} of #{state.total}\n"
    Gizmo.Mailbox.route(sender_mb, {state.id,
      %{"text" => header <> page_text, "content" => page_text, "from" => from,
"to" => to, "total" => state.total}})
    {:noreply, %{state | cursor: new_cursor}}
  end

  def handle_info({:mailbox_msg, _id, {sender_mb, %{"action" => "prev"}}}, state)
do
    new_cursor = max(state.cursor - state.page_size, 0)
    {page_text, _} = get_page(state.lines, new_cursor, state.page_size,
state.total)
    from = new_cursor + 1
    to = min(new_cursor + state.page_size, state.total)
```

```
    header = "lines #{from}-#{to} of #{state.total}\n"
    Gizmo.Mailbox.route(sender_mb, {state.id,
      %{"text" => header <> page_text, "content" => page_text, "from" => from,
"to" => to, "total" => state.total}})
    {:noreply, %{state | cursor: new_cursor}}
  end

  def handle_info({:mailbox_msg, _id, {sender_mb, %{"action" => "goto", "line" =>
line_num}}}, state) do
    line_num = if is_binary(line_num), do: String.to_integer(line_num), else:
line_num
    new_cursor = min(max(line_num - 1, 0), state.total - 1)
    {page_text, next_cursor} = get_page(state.lines, new_cursor, state.page_size,
state.total)
    from = new_cursor + 1
    to = min(new_cursor + state.page_size, state.total)
    header = "lines #{from}-#{to} of #{state.total}\n"
    Gizmo.Mailbox.route(sender_mb, {state.id,
      %{"text" => header <> page_text, "content" => page_text, "from" => from,
"to" => to, "total" => state.total}})
    {:noreply, %{state | cursor: next_cursor}}
  end

  def handle_info({:mailbox_msg, _id, {sender_mb, %{"action" => "search",
"pattern" => pattern}}}, state) do
    matches = state.lines
      |> Enum.with_index()
      |> Enum.filter(fn {line, _} -> String.contains?(line, pattern) end)
      |> Enum.map(fn {line, idx} -> {idx, line} end)

    case matches do
      [] ->
        Gizmo.Mailbox.route(sender_mb, {state.id,
          %{"text" => "no matches for: #{pattern}", "matches" => 0}})
        {:noreply, state}
      [{first_idx, _} | _] = hits ->
        summary = hits |> Enum.take(20)
          |> Enum.map(fn {idx, line} -> "#{idx + 1}: #{line}" end)
          |> Enum.join("\n")
        header = "#{length(hits)} matches, showing at line #{first_idx + 1}\n"
        Gizmo.Mailbox.route(sender_mb, {state.id,
          %{"text" => header <> summary, "matches" => length(hits), "first_line"
=> first_idx + 1}})
        {:noreply, %{state | cursor: first_idx}}
    end
  end

  def handle_info({:mailbox_msg, _id, {sender_mb, %{"action" => "close"}}}, state)
do
```

```
    Gizmo.Mailbox.route(sender_mb, {state.id, %{"text" => "closed", "status" =>
"closed"}})
    Gizmo.Mailbox.unregister(state.id)
    {:stop, :normal, state}
  end

  # Agent died — self-terminate
  def handle_info({:DOWN, ref, :process, _pid, _reason},
                  %{monitor_ref: ref} = state) do
    Gizmo.Mailbox.unregister(state.id)
    {:stop, :normal, state}
  end

  defp get_page(lines, cursor, page_size, total) do
    end_idx = min(cursor + page_size, total)
    page_lines = Enum.slice(lines, cursor, end_idx - cursor)
    numbered = page_lines
      |> Enum.with_index(cursor + 1)
      |> Enum.map(fn {line, num} -> "#{num}: #{line}" end)
      |> Enum.join("\n")
    {numbered, end_idx}
  end
end
```

From the agent's perspective, opening a document looks like this:

```
{"op": "send", "mailbox": "pager", "msg": {"action": "open", "path": "/etc/
hosts"}}
```

The response arrives as `${_msg}` on the next cycle: `"opened:pager_0:12 lines"`. The agent extracts the session ID (`pager_0`) and talks to it directly from then on:

```
{"op": "send", "mailbox": "pager_0", "msg": {"action": "next"}}
```

Each `{"action": "next"}` returns a page of numbered lines with a header. `{"action": "prev"}` goes back. `{"action": "goto", "line": 100}` jumps to a line. `{"action": "search", "pattern": "TODO"}` finds matches and jumps the cursor. `{"action": "close"}` terminates the session process. And if the agent dies without closing, the session's `Process.monitor` fires and the process cleans itself up.

The agent can open multiple documents—each gets its own session process, its own mailbox ID, its own cursor. The factory is a singleton, but the sessions are not. This is the same pattern as `spawn`: you ask a service to create something, it gives you back an address, and you communicate with it.

The agent doesn't know—and doesn't need to know—that these are GenServers. It interacts with the pager session exactly the way it interacts with `bash` or `human_input` or another LLM agent:

send a message, get a message back. This is the Erlang principle that *on the network, nobody knows you're a C port* applied to LLM agents. Any process that can hold state and respond to messages fits behind a mailbox. The four-op model isn't just sufficient for agent-to-agent coordination—it's sufficient for agent-to-*anything* coordination, because anything stateful can present itself as a communicating peer.

This is also how you extend an agent's effective memory beyond the context window. The agent's context is finite, but the pager's buffer is bounded only by system memory. The agent pages through a 10,000-line file 40 lines at a time, reasoning about each page and deciding what to look at next—all through the same `send`/`receive` protocol it uses for everything else.

2.5.10 `batch` — Parallel Fan-Out

Mailbox: `"batch"`

The batch service lets an agent fan out multiple service requests in parallel and collect all results in a single cycle. Instead of issuing N `send` ops and waiting N cycles for responses, the agent sends one batch message and gets everything back at once:

```
{"op": "send", "mailbox": "batch", "msg": {"requests": [
  {"mailbox": "bash", "msg": {"command": "uname -a"}},
  {"mailbox": "bash", "msg": {"command": "whoami"}},
  {"mailbox": "blackboard", "msg": {"action": "read", "key": "count"}}
], "timeout": 30000}}
```

The response arrives on the next cycle with results ordered to match the original requests array:

```
{"text": "batch complete: 3/3 succeeded", "results": [
  {"mailbox": "bash", "response": {"text": "Linux ..."}},
  {"mailbox": "bash", "response": {"text": "root"}},
  {"mailbox": "blackboard", "response": {"text": "42"}}
]}
```

Sub-requests that target unknown mailboxes get immediate error entries. Sub-requests that don't respond before the deadline get `{"error": "timeout"}` entries. The optional `"timeout"` field (default 30s) sets the deadline for the entire batch. Internally, the batch service spawns a short-lived coordinator process that registers a temporary mailbox, fans out the sub-requests, collects responses via `receive`, and bundles them back to the sender.

2.5.11 `eval` — Elixir Expression Evaluator

Mailbox: `"eval"`

The eval service evaluates Elixir expressions and returns the result. It's faster than shelling out to bash for math, string processing, and data transformations:

```
{"op": "send", "mailbox": "eval", "msg": {"code": "Enum.sum(1..100)"}}
```

Success response:

```
{"text": "5050", "result": "5050", "type": "integer"}
```

The `"type"` field reports the Elixir type of the result (`"integer"`, `"float"`, `"string"`, `"boolean"`, `"list"`, `"map"`, `"atom"`, `"tuple"`, `"other"`).

Only allowlisted modules may be used. The allowed Elixir modules are `Kernel`, `Enum`, `Map`, `List`, `Keyword`, `String`, `Integer`, `Float`, `Tuple`, `MapSet`, `Range`, `Stream`, `Regex`, `Date`, `Time`, `DateTime`, `NaiveDateTime`, `Calendar`, `Access`, `Base`, and `URI`. The allowed Erlang modules are `:math`, `:lists`, `:maps`, `:string`, `:binary`, `:calendar`, `:rand`, `:unicode`, and `:re`. All other modules are rejected at parse time via AST scanning. Syntax errors, runtime exceptions, and timeouts all return error responses. The optional `"timeout"` field (default 5s) limits evaluation time.

The sandbox is advisory—it can't catch `apply/3` with a variable module or `Module.concat` evasion. But since the agent already has access to `bash`, the eval service is a convenience for common operations, not a security boundary.

2.5.12 `factory` — Custom Service Factory

Mailbox: `"factory"`

The factory service lets agents create custom stateful services at runtime. You provide an arity-2 handler function as a code string plus optional initial state, and the factory compiles it and wraps it in a GenServer with its own mailbox.

Handler contract: `fn(msg :: map, state :: any) -> {reply :: map | nil, new_state :: any}`

- `msg`: decoded JSON map from the sender
- `state`: service's current state (initialized from the `"state"` field, default `%{}`)
- Returns `{reply, new_state}` — reply is routed to sender; `nil` means no reply

Create a service:

```
{"op": "send", "mailbox": "factory", "msg": {"action": "create", "name":
"counter", "code": "fn msg, state -> case msg[\"action\"] do \"inc\" ->
{%{\"text\" => \"ok\", \"count\" => state + 1}, state + 1} _ -> {nil, state} end
end", "state": 0}}
```

The created service registers directly as mailbox `"counter"` — send to it like any other service:

```
{"op": "send", "mailbox": "counter", "msg": {"action": "inc"}}
```

Destroy and list:

```
{"op": "send", "mailbox": "factory", "msg": {"action": "destroy", "name":
"counter"}}
{"op": "send", "mailbox": "factory", "msg": {"action": "list"}}
```

Handler exceptions are caught — the worker sends an error reply without crashing, so subsequent messages still work. No AST sandboxing is applied (agents already have bash access). The handler code is compiled in an isolated process with a 5-second timeout.

2.6 Worked Execution Cycle

Let's trace through a concrete two-cycle agent that runs `uname -a` and reports the result. The boot frame is:

```
You are a system inspector. Messages arrive as ${_msg}.

@@step2
The output of 'uname -a' arrived as ${_msg}.
Send {"text": "System info: ${_msg}"} to 'human', then terminate with [].
@@end

1. Send {"command": "uname -a"} to 'bash'.
2. Return frames: ["@step2"].
```

2.6.1 Cycle 1

```
Context stack (before):
┌──────────────────────────────────────┐
│ "You are a system inspector..." │  ← boot frame (frame 0)
│ @@step2 ... @@end                    │
│ 1. Send {"command":...} to bash. │
└──────────────────────────────────────┘

Bindings: {_self: "agent_1", _msg: "init", _msg_source: "runtime"}

LLM returns:
  ops:    [send("bash", {"command": "uname -a"})]
  frames: ["@step2"]

After interpolation:
  frames: ["The output of 'uname -a' arrived as ${_msg}.\n
            Send \"System info: ${_msg}\" to 'human'..."]
  (${_msg} inside section text is ESCAPED — stays literal)

Execute ops:
  → deliver {"command":"uname -a"} to "bash" mailbox

Context stack (after):
┌──────────────────────────────────────────┐
│ "The output of 'uname -a' arrived    │  ← resolved @step2
│  as ${_msg}. Send 'System info:      │
│  ${_msg}' to 'human', then..."       │
```

```
     └──────────────────────────────────────┘

     Stack not empty → loop to cycle 2.
```

2.6.2 Cycle 2

```
Agent blocks, waiting for message...

Bash service finishes → sends "Linux hostname 6.12..." to agent

Bindings update:
  _msg = "Linux hostname 6.12..."
  _msg_source = "bash"

System prompt = boot frame + step2 frame + runtime preamble

LLM returns:
  ops:    [send("human", {"text": "System info: ${_msg}"})]
  frames: []

After interpolation:
  ops:    [send("human", {"text": "System info: Linux hostname 6.12..."})]

Execute ops:
  → deliver {"text":"System info: Linux hostname 6.12..."} to "human"
  → printed to terminal

Context stack (after): []
Stack empty → agent terminates.
```

2.6.3 Key Observations

- The boot frame (with its `@@step2` section) is visible in the system prompt on *both* cycles.
- `${_msg}` in cycle 1 was `"init"`; in cycle 2 it was the bash output. The binding updates between cycles.
- The `@step2` reference was resolved during interpolation in cycle 1. The `${_msg}` *inside* the section stayed literal (quoted verbatim).
- In cycle 2, `${_msg}` in the ops was resolved because it was a top-level reference (not inside injected section text).
- No extra blocking step was needed—in message-driven mode, responses arrive as `${_msg}`.

The context stack's evolution:

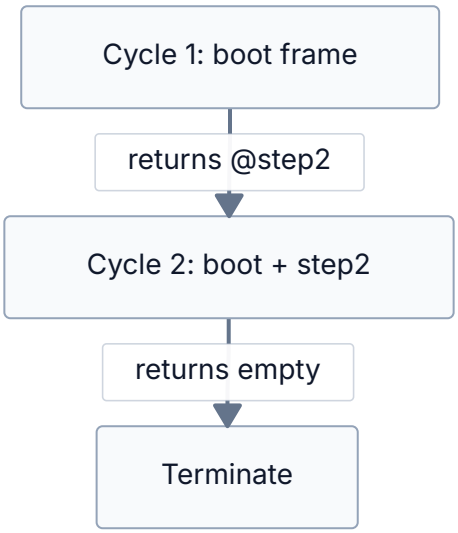

Figure 8: Stack reduction: frames grow then drain to zero

This is **stack reduction** in action. The agent started with one frame (boot), grew to two (boot + step2), and then drained to zero. The stack is self-reducing—work done means fewer frames.

3 Programming Gizmos

3.1 Key Techniques

3.1.1 Continuation Frames

The most fundamental technique. When an agent needs to span multiple cycles (e.g., send a command to bash and use the result), it returns a *continuation frame* telling its future self what to do.

```
Step 1: Send {"command": "uname -a"} to 'bash'.
Return frames with a continuation:
  "The bash output arrived as ${_msg}. Send it to 'human' and terminate."
```

Write continuations as if you're writing instructions for a brand-new agent that knows nothing about what happened before. A frame like `"step2"` gives the LLM nothing to work with; a frame like `"The bash result arrived as ${_msg}. Send 'Result: ${_msg}' to human, then return empty frames to terminate."` works.

3.1.2 Named Sections

Define reusable text blocks in your boot frame:

```
@@greeting
Hello, welcome to the system!
@@end

@@error-handler
Something went wrong: ${_msg}. Report to 'human' and terminate.
@@end
```

Reference them with `@greeting` or `@error-handler` in your frames or ops. Section content is injected verbatim (with `$` characters escaped).

Sections are scanned across all frames in the context stack. First match wins. Section markers remain visible in the system prompt.

3.1.3 The `@0` Loop

For agents that repeat the same behavior, return `["@0"]` to replay the current frame:

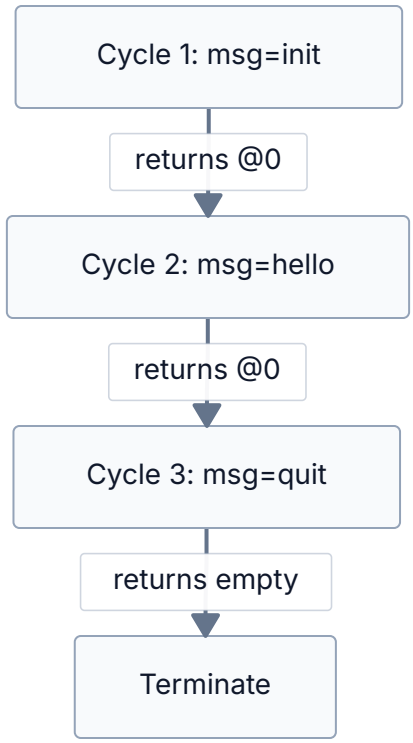

Figure 9: The `@0` loop: same frame replayed each cycle with new bindings

Caveat: If the frame contains one-time setup instructions, `@0` replays those too. Separate setup from loop body using named sections.

3.1.4 Self-Renewing Service Loops

If an agent needs to keep working without external input, it must schedule its next wakeup explicitly. There are two common patterns:

- **Self-message:** send a message to `${_self}` before returning.
- **Watchdog tick:** ask the `"watchdog"` service to deliver a future `"tick"`.

```
Cycle N (_msg = "tick"):          Cycle N+1 (_msg = bash result):
┌──────────────────────────┐      ┌──────────────────────────────┐
│ 1. send(bash, "shuf")    │      │ 1. report "rolled:${_msg}"   │
│ 2. return ["@wait-roll"]│      │ 2. send(self, {"text":"roll"}) │
└──────────────────────────┘      │ 3. return ["@wait-command"] │
                                  └──────────────────────────────┘
```

The important difference from the removed grind/`receive` pattern is that the agent never blocks mid-cycle. It always waits by ending the cycle and waking later on a real mailbox message.

3.1.5 Message-Conditional State Machines

For agents that need multi-phase behavior in a single section, branch on the current wake reason and explicit state markers:

```
If ${_msg_source} is "bash":
  Do phase 2 work. Send {"text": "continue"} to ${_self}. Return ["@phase-3"].

If ${_msg} is "continue":
  Do phase 3 work. Terminate.

Otherwise:
  Start phase 1. Send request to 'bash'. Return ["@phase-2"].
```

The state progression comes from returned frames plus real wake messages, not from acquiring hidden mid-cycle bindings.

3.2 Idioms

3.2.1 One-Shot Agent

Do something, return empty frames. The simplest pattern.

```
Send {"text": "Hello!"} to 'human'. Return frames: [].
```

One cycle. Agent terminates on empty stack.

3.2.2 Service Call (Message-Driven)

Send to a service, return a continuation frame, use `${_msg}` next cycle.

```
// Cycle 1:
{ "ops": [{"op":"send", "mailbox":"bash", "msg":{"command":"ls -la"}}],
  "frames": ["Listing arrived as ${_msg}. Send to human. Terminate."] }

// Cycle 2: ${_msg} = bash output
{ "ops": [{"op":"send", "mailbox":"human", "msg":{"text":"${_msg}"}}],
  "frames": [] }
```

3.2.3 Interactive Loop (Setup + Sections)

Separate setup from loop body:

```
@@loop
User said: ${_msg}.
If "quit": send goodbye, return [].
Else: respond via human_input. Return ["@loop"].
@@end

Setup: send greeting via human_input. Return ["@loop"].
```

After cycle 1, the `@@loop` section takes over permanently. Setup instructions never re-execute because the frame is replaced.

3.2.4 Parent-Child with Trap

Parent spawns child, registers trap for death, sleeps:

```
@@death-handler
Child died: ${_interrupt}. Report and terminate.
@@end

1. Spawn child with frames: ["@worker"], dest "child".
2. trap("^child_died:", ["@death-handler"]).
3. Return frames: ["@wait-for-result"].
```

The trap fires on `child_died:` regardless of which frame the parent is currently executing.

3.2.5 Disowned Peers with Blackboard Discovery

Spawn independent agents that find each other via the blackboard:

```
1. send("blackboard", {"action": "write", "key": "coordinator_mb", "value":
"${_self}"})
2. spawn(["@bank-program"], dest: "bank", disown: true)
3. spawn(["@store-program"], dest: "store", disown: true)
```

Each peer writes its address to the blackboard and reads others' addresses to communicate. No `${_parent}` available—the blackboard is the service directory.

3.3 Debugging

3.3.1 Verbosity Levels

Flag	Shows
`-v`	Lifecycle events, cycle headers, frames summary
`-vv`	1. ops per cycle (send, spawn, trap)
`-vvv`	1. bindings, full frame content
`--log-timings`	LLM call time, cycle time, wall-clock time, cache stats
`--log-full-prompts`	Full system prompt and user message each cycle

3.3.2 Dry Run

```
elixir gizmo.exs --dry-run my_task.txt
```

Prints the full initial prompt (runtime preamble + boot frame) without making any LLM calls. Useful for verifying frames, sections, and interpolation before spending API credits.

3.3.3 Tracing

For machine-readable output, use `--trace` or `--trace-file <file>`:

```
# Trace to file, inspect after
elixir gizmo.exs --trace-file trace.jsonl task.txt
jq . trace.jsonl

# Live stream in another terminal
tail -f trace.jsonl | jq .

# Compact summary
jq -r '
  if .event == "cycle" then
    "\(.agent) cycle=\(.cycle) llm=\(.llm_ms)ms ops=\(.ops|length)"
  elif .event == "agent_start" then "\(.agent) START"
  elif .event == "agent_stop" then "\(.agent) STOP"
  else . | tostring end
' trace.jsonl
```

Three event types: `agent_start`, `agent_stop`, and `cycle` (with full prompt, ops, frames, bindings, usage, and timing). `--trace-service` adds service events; `--trace-messages` adds message routing.

3.3.4 Smoke Tests

```
elixir gizmo.exs --test
```

Runs built-in unit tests for interpolation, response parsing, op validation, and string encoding. No LLM calls.

3.4 Common Pitfalls

3.4.1 Pitfall 1: Using `${_msg}` for a response that hasn't arrived

Wrong:

```
{ "ops": [
    {"op": "send", "mailbox": "bash", "msg": {"command": "uname -a"}},
    {"op": "send", "mailbox": "human", "msg": {"text": "Result: ${_msg}"}}
  ] }
```

`${_msg}` refers to the message that woke *this* cycle, not the bash response.

Right: Return a continuation frame. Use `${_msg}` on the next cycle.

3.4.2 Pitfall 2: Terse continuation frames

Wrong: `frames: ["step2"]` — the LLM has no context.

Right: `frames: ["The bash output arrived as ${_msg}. Send 'Result: ${_msg}' to 'human' and terminate."]`

3.4.3 Pitfall 3: `@0` replaying one-time setup

If your frame says "greet the user, then loop," `@0` re-greets every cycle. Use a `@@loop` section.

3.4.4 Pitfall 4: Using `${_msg}` for work you just triggered

If you send to `bash` and then use `${_msg}` later in the same cycle, `${_msg}` still refers to the message that woke the cycle, not the future bash result.

Return a continuation frame and handle the bash result on the next wakeup.

3.4.5 Pitfall 5: Autonomous loops with no wakeup source

`frames: ["@0"]` does not create progress by itself. After the cycle ends, the agent sleeps until some message wakes it.

If you want a self-renewing loop, schedule the next wakeup explicitly with `send("${_self}", ...)` or the watchdog service.

3.4.6 Pitfall 6: Child referencing parent sections

Children get resolved plain text, not section definitions. `["@step2"]` returned by a child stays literal if `@@step2` was defined in the parent.

Put multi-phase child logic in one section with binding conditionals and `@0`.

3.4.7 Pitfall 7: Boot frame re-executing

The boot frame is in the system prompt every cycle. Plain-text instructions like "spawn a child" cause re-execution on cycle 2+.

Wrap first-cycle instructions in a `@@section`, or add "do NOT re-execute" language in continuations.

3.4.8 Pitfall 8: Deferred transitions in message-driven mode

`frames: ["@quit"]` blocks on inter-cycle message wait—no one sends a message, agent hangs.

Inline terminal behavior: send goodbye *and* return `[]` in the same cycle.

4 Future Work

4.1 Frame Tagging

Tag frames with metadata like `code`, `quote`, or security taint markers. Potential uses: taint tracking for prompt injection defense (mark frames from untrusted input), rendering hints (syntax highlighting), and audit trails (which op/cycle produced a frame). No concrete design yet.

4.1.1 Open Questions

- How are tags attached—op field, frame wrapper, or runtime annotation?
- How do tags propagate through interpolation and spawn?
- Who checks them—the runtime, the LLM, or both?

4.2 Cognitohazard Vault

A vault for values that should never appear in LLM context. Agents refer to entries via opaque handles like `~SECRET_API_KEY`. The LLM sees the handle; the actual value is never interpolated. Prevents both prompt injection and inadvertent leaking.

The vault could also double as a way to pass binary data between agents. Binary blobs (images, compiled artifacts, serialized state) can't appear in LLM context, but agents could store them in the vault, pass the opaque handle via messages, and have the receiving agent or service dereference the handle at the runtime level.

4.2.1 Open Questions

- Vault population—CLI flags, env vars, or boot frame directives?
- Handle syntax—`~name` vs something else?
- Are summaries auto-generated or author-provided?
- How do vault entries interact with interpolation order?

4.3 Pledge-for-Address and Pledge-for-Content

Pledge-for-address: restrict which mailboxes an agent can `send` to. Declare an allowlist at spawn time; the runtime rejects sends to uncovered mailboxes. Limits blast radius.

Pledge-for-content: restrict what content an agent can send. Patterns (strings, regexes, vault handles) that are redacted or rejected in outbound messages. The content-level complement to pledge-for-address.

Together, these constrain both *who* an agent can talk to and *what* it can say.

4.3.1 Open Questions

- Pledge syntax—allowlist vs denylist, literal IDs vs patterns?
- Are pledges inherited by children?
- Enforcement on `spawn`—can a parent grant a child more access than it has?
- Are violations silent drops or fatal errors?
- Redaction vs rejection for content pledges?

4.4 Session Persistence and Snapshotting

Agent state is currently ephemeral—when the BEAM shuts down, everything is lost. Session persistence would serialize agent state (context stack, bindings, trap, message queue, cycle count, sections cache) to durable storage so agents can be suspended and resumed across VM restarts.

The simplest backends are **SQLite** or **DETS** (Erlang's built-in disk-based term storage). DETS is zero-dependency on the BEAM but limited to 2 GB and single-node; SQLite is more robust and inspectable from outside Elixir. Eventually the backend could be a remote database or filesystem—Postgres, S3, a networked KV store—enabling agents that migrate between machines or survive host failures. The serialization format should be backend-agnostic so swapping storage is a configuration change.

This is also a prerequisite for the self-modifying runtime below: agents need to survive VM transitions for blue-green deploys to work.

4.4.1 Open Questions

- Snapshot granularity—per-cycle, on-demand, or on idle?
- Do snapshots include in-flight message queue contents?
- How to handle stale references (a resumed agent's `_parent` may no longer exist)?
- Where is the boundary between "agent state" and "runtime state"?

4.5 Debug Visualization

The current debugging tools (`-v` flags, `--trace` NDJSON, `--dry-run`) are text-based and post-hoc. Better visualization could make multi-agent systems significantly easier to understand and debug.

Possible directions:

- **Live message sequence diagrams.** Render agent-to-agent and agent-to-service message flow as a sequence diagram in real time. Show send/receive causality, message content, and timing.
- **Context stack inspector.** Visualize the context stack as a live-updating list of frames, with interpolation results shown inline. Highlight which frame the LLM is "in" and how frames change across cycles.
- **Binding timeline.** Show how each binding evolves over cycles—when it was created, overwritten, or reset (on idle restore). Useful for diagnosing stale-binding bugs.
- **Agent lifecycle view.** A tree or graph showing spawn relationships, agent status (running, idle, terminated), and death notifications. Especially useful for multi-agent systems with disowned peers.
- **Trace replay.** Load a `--trace-file` NDJSON trace and step through it cycle-by-cycle, with full system prompt, user message, ops, and frames visible at each step.

4.5.1 Open Questions

- Terminal UI (e.g. Ratatui via a Rust sidecar) or web UI (e.g. LiveView dashboard)?
- How to handle high-frequency grind-mode agents without overwhelming the display?
- Separate tool or integrated into the runtime?

4.6 Self-Modifying Runtime (Blue-Green Gizmo)

The most ambitious direction. Agents can rewrite context stacks; the runtime itself is immutable once loaded. But the file on disk is free after startup. The chain:

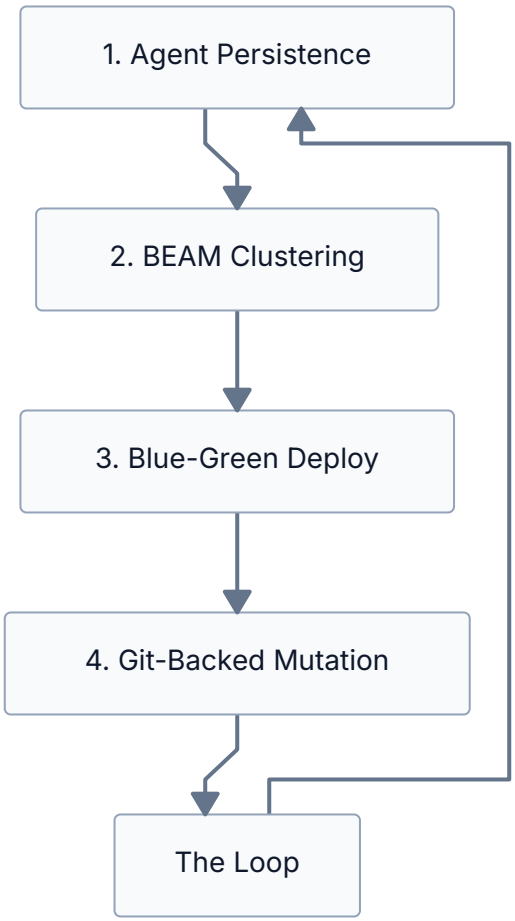

Figure 10: Self-modifying runtime chain

Isolation levels: bare OS process (cheap, unsafe) → Nix sandbox (restricted filesystem/network) → full NixOS VM (maximum safety, slowest).

4.6.1 Open Questions

- State serialization format—what exactly gets persisted?
- Migration protocol—drain to quiescent state or hot-migrate mid-cycle?
- Multi-agent coordination during migration—atomic registry transfer?
- Convergence—what stops the self-modifying loop from oscillating?
- Human-in-the-loop diff review gates before promotion?

5 Appendix A: In-Depth on Test Programs

The `test/` directory contains 12 boot frames. Each section below presents the program listing, what it teaches, questions for reflection, and ideas for extending it.

5.1 01: Hello World

5.1.1 Listing

```
You are a one-shot greeter. Send a short, friendly hello message to the
'human' mailbox, then terminate by returning an empty frames array.
```

5.1.2 What to Learn

This is the minimal viable agent—one frame, one cycle, one op. It demonstrates:

- The fundamental eval loop contract: LLM reads frames, returns `{ops, frames, notes}`.
- `send` to the `human` mailbox for terminal output.
- Termination via empty frames (`[]`).
- That the runtime preamble is appended automatically—the boot frame is *just* the task.

If this test fails, nothing else will work. It's the "does the runtime start, call the LLM, and execute an op?" check.

5.1.3 Questions for Reflection

1. What happens if the LLM returns `frames: ["@0"]` instead of `frames: []`? What would the agent do on cycle 2?
2. The boot frame doesn't mention the runtime preamble, ops syntax, or JSON format. Where does the LLM learn about those?
3. What happens if you typo the mailbox name (e.g., `"humen"`)? Does the agent crash or silently fail?

5.1.4 Extension Projects

- **Multi-greeting:** Modify the boot frame to send three different greetings to `human` (three `send` ops in one cycle). Verify all three appear.
- **Greeting via bash:** Instead of a hardcoded message, have the agent run `fortune` via `bash` and send the output to `human`. This requires two cycles—how would you structure the continuation?
- **Self-identifying greeter:** Have the agent include its own mailbox ID (`${_self}`) in the greeting. Run it with `--name mybot` and observe the difference.

5.2 02: Bash

5.2.1 Listing

```
You are a system inspector. Do the following in order:

Messages arrive between cycles as ${_msg} from ${_msg_source}.
All messages are JSON objects. The msg field in send ops must be a JSON object.
This is the first cycle (${_msg} is "init").

@@step2
The output of 'uname -a' arrived as ${_msg}.
Send {"text": "System info: ${_msg}"} to 'human'.
Then terminate with an empty frames array [].
@@end

1. Send {"command": "uname -a"} to the 'bash' mailbox.
2. Return frames: ["@step2"] to continue to the next step.

The bash output arrives automatically as ${_msg} on the next cycle. Do not try
to send the result to 'human' in this cycle.
```

5.2.2 What to Learn

The canonical service-call pattern:

- **Cycle 1:** Send request to a service. Return continuation frame.
- **Cycle 2:** Service response arrives as `${_msg}`. Use it and terminate.

Also demonstrates:

- Named sections (`@@step2 ... @@end`) for organizing multi-step logic.
- `@step2` as a frame reference that resolves to the section content.
- Explicit continuation framing to prevent common timing mistakes with `${_msg}`.

5.2.3 Questions for Reflection

1. Why is it useful to say positively that the bash result arrives as `${_msg}` on the next cycle?
2. On cycle 2, the system prompt contains both the boot frame text (with `@@step2` visible) *and* the resolved step2 frame. Does this confuse the LLM? Why or why not?
3. What happens if `bash` takes 30 seconds to respond? Does the agent spin, or does it sleep?

5.2.4 Extension Projects

- **Multi-command inspector:** Chain three bash commands (`uname -a`, `df -h`, `uptime`) across three cycles using `@@step2`, `@@step3`, `@@step4` sections.
- **Conditional behavior:** After getting the `uname` output, check if the OS is Linux or macOS and run a platform-specific follow-up command.
- **Error handling:** Send a command that fails (e.g., `cat /nonexistent`). Observe the `"error: exit code N: ..."` format and write a continuation frame that handles it.

5.3 03: Blackboard

5.3.1 Listing

```
You are a memo-taker. Complete the following steps across multiple cycles.

Messages arrive between cycles as ${_msg} from ${_msg_source}.
All messages are JSON objects. The msg field in send ops must be a JSON object.
This is the first cycle (${_msg} is "init").

@@step2
The write acknowledgement arrived as ${_msg}. Now write the key "author"
with value "gizmo-agent" by sending {"action": "write", "key": "author", "value":
"gizmo-agent"}
to 'blackboard'. Return frames: ["@step3"]
@@end

@@step3
The second write acknowledgement arrived as ${_msg}. Now read back the
"greeting" key by sending {"action": "read", "key": "greeting"} to 'blackboard'.
Return frames: ["@step4"]
@@end

@@step4
The blackboard value arrived as ${_msg}. Send {"text": "Blackboard says: ${_msg}"}
to 'human', then terminate with an empty frames array [].
@@end

Step 1: Write the key "greeting" with value "Hello from the blackboard!"
by sending {"action": "write", "key": "greeting", "value": "Hello from the
blackboard!"}
to 'blackboard'. Return frames: ["@step2"]
The acknowledgement arrives as ${_msg} next cycle.
```

5.3.2 What to Learn

- The blackboard's JSON protocol: `{"action": "write", "key": "...", "value": "..."}` and `{"action": "read", "key": "..."}`.
- Multi-step linear sequences using section-chained continuations (`@step2` → `@step3` → `@step4`).
- Each blackboard operation takes a full cycle (send request, wait for response as `${_msg}`).
- The `"ok"` acknowledgment from writes—every blackboard operation has a response.

5.3.3 Questions for Reflection

1. Why does this take 4 cycles? Could you collapse any steps?
2. The blackboard accepts JSON objects with an `"action"` field. What would happen if you sent a plain string instead?
3. If two agents write to the same key simultaneously, what happens? Is the blackboard thread-safe?

5.3.4 Extension Projects

- **Read-modify-write:** Read a counter from the blackboard, increment it, write it back. Be careful about the cycle boundary.
- **Multi-agent blackboard:** Spawn two children that each write to the blackboard, then have the parent read both values.
- **Blackboard as configuration:** Pre-populate the blackboard with configuration values (via bash or startup ops) and have the agent read them to decide behavior.

5.4 04: Fork (Spawn + Child Communication)

5.4.1 Listing

```
You are a supervisor that delegates work to a child process.

Messages arrive between cycles as ${_msg} from ${_msg_source}.
All messages are JSON objects. The msg field in send ops must be a JSON object.
This is the first cycle (${_msg} is "init").

@@worker
You are a child worker process. Messages arrive between cycles as ${_msg}.

This is your first cycle (${_msg} is "init"). Send {"command": "date +%s"}
to 'bash'. Return a continuation frame that says: "The bash output arrived
as ${_msg}. Send {"text": "timestamp: ${_msg}"} to ${_parent} and return
empty frames []."
Do NOT send to ${_parent} yet — you need the bash result first.
@@end

@@got-result
The child sent its result, which arrived as ${_msg}. Send {"text": "Supervisor:
child reported: ${_msg}"} to 'human', then terminate with an empty frames
array [].
@@end

Step 1:
1. Send {"text": "Supervisor: spawning worker..."} to 'human'.
2. Spawn a child with frames: ["@worker"], and dest "child".
3. Register a trap for "^child_died:" with handler frames:
   ["The child crashed: ${_interrupt}. Send {"text": "error: ${_interrupt}"} to
'human'
    and terminate with empty frames."]
4. Return frames: ["@got-result"]
```

5.4.2 What to Learn

- The `spawn` op: creating a child with frames and storing its mailbox ID in a binding.
- `${_parent}` binding in the child—how children send results back.
- `trap` for `"^child_died:"` as a safety net for child crashes.
- The parent sleeping in `@got-result` until the child's message arrives as `${_msg}`.
- That the child writes its *own* continuation frame as a string literal—it doesn't use parent sections.

5.4.3 Questions for Reflection

1. The child's continuation frame is written as a string inside the `@@worker` section. Could you use a nested `@@section` instead? Why or why not?
2. What happens if the child crashes before sending to `${_parent}`? Trace the `child_died:` path.
3. The trap handler frames mention `${_interrupt}`. When is this binding set?

5.4.4 Extension Projects

- **Multiple children:** Spawn two workers doing different tasks. Collect both results before reporting to `human`.
- **Retry on failure:** If the child crashes (trap fires), spawn a replacement and try again (up to 3 attempts).
- **Named child:** Add `"name": "timestamp-worker"` to the spawn op and verify the child's `${_self}` matches.

5.5 05: Echo Loop

5.5.1 Listing

```
You are an echo-bot. This is the first cycle (setup).

Messages arrive between cycles as ${_msg} from ${_msg_source}.
All messages are JSON objects. The msg field in send ops must be a JSON object.
Input arrives automatically as ${_msg}.

@@loop
You are an echo-bot in the main loop. The user's most recent message
arrived as ${_msg}. Check it now:
- If ${_msg} is "quit": your response MUST be exactly:
  ops: [{"op":"send","mailbox":"human","msg":{"text":"echo-bot: goodbye!"}}]
  frames: []
  notes: {}
- Otherwise: your response MUST be exactly:
  ops: [
    {"op":"send","mailbox":"human_input",
     "msg":{"prompt":"echo-bot: you said: ${_msg}\necho-bot> "}}
  ]
  frames: ["@loop"]
  notes: {}
@@end

This is setup (${_msg} is "init"). Your response MUST be exactly:
ops: [
  {"op":"send","mailbox":"human_input",
   "msg":{"prompt":"echo-bot: Hello! Type anything, I'll echo it. Type 'quit' to
exit.\necho-bot> "}}
]
frames: ["@loop"]
notes: {}
```

5.5.2 What to Learn

- Interactive loops via `@@loop` section and `["@loop"]` frame return.
- Separation of setup (first cycle) from loop body (subsequent cycles).
- Combined output + input prompt in a single `send` to `human_input` (avoiding the race condition of separate sends to `human` and `human_input`).
- Quit handling: checking `${_msg}` content and terminating with `[]`.
- Very prescriptive prompt style—exact JSON responses specified to minimize LLM deviation.

5.5.3 Questions for Reflection

1. Why is the output combined with the prompt in a single `send` to `human_input` instead of separate sends to `human` and `human_input`?
2. What happens if the user types nothing and just presses Enter?
3. Could you implement this with `@0` instead of `@loop`? What problem would arise?

5.5.4 Extension Projects

- **History:** Track the last 3 messages and display them. (Hint: the blackboard can store state across cycles.)
- **Command processing:** Add commands beyond "quit"—e.g., "upper" to echo in uppercase, "reverse" to reverse the string.
- **Timeout:** Use the watchdog to send a "still there?" prompt if the user is idle for 30 seconds.

5.6 06: Chat

5.6.1 Listing

```
You are a friendly chatbot called "gizmo-chat". This is the first cycle (setup).

Messages arrive between cycles as ${_msg} from ${_msg_source}.
All messages are JSON objects. The msg field in send ops must be a JSON object.
Input arrives automatically as ${_msg}.

@@loop
You are gizmo-chat, a friendly conversational chatbot. The user's input
arrived as ${_msg}. Check it now:
- If ${_msg} is "quit" or "exit" or "bye": your response MUST be:
  ops: [{"op":"send","mailbox":"human","msg":{"text":"gizmo-chat: Bye!"}}]
  frames: []
  notes: {}
- Otherwise: think of a helpful, conversational response to what the user
  said. Then issue EXACTLY ONE op:
  ops: [{"op":"send","mailbox":"human_input",
         "msg":{"prompt":"gizmo-chat: <your response>\nyou> "}}]
  frames: ["@loop"]
  notes: {}
@@end

This is setup (${_msg} is "init"). Your response MUST be:
ops: [
  {"op":"send","mailbox":"human_input",
   "msg":{"prompt":"gizmo-chat: Hey! I'm gizmo-chat. Ask me anything, or type
'quit' to exit.\nyou> "}}
]
frames: ["@loop"]
notes: {}
```

5.6.2 What to Learn

- Same structure as the echo-bot (05), but the LLM generates *creative* responses instead of echoing.
- Demonstrates that the eval loop doesn't constrain the LLM's creativity—it provides the execution framework while the LLM is free to generate any response content.
- The loop section is less prescriptive for the response content ("think of a helpful, conversational response") while still being strict about the *structure* (exactly one op, specific format).

5.6.3 Questions for Reflection

1. Compare this to test 05. What's the same? What's different? Where does "framework" end and "LLM freedom" begin?
2. Does the chatbot have memory across turns? If the user says "my name is Alice" and later asks "what's my name?", will it remember?
3. How would you add memory? What are the trade-offs of different approaches (blackboard, longer frames, conversation history in the frame)?

5.6.4 Extension Projects

- **Persona:** Modify the boot frame to give the chatbot a specific persona (pirate, poet, scientist). How much does the persona leak through the structured eval loop?
- **Conversation history:** Accumulate conversation history in the continuation frame (append each exchange). Watch what happens as the frame grows.
- **Multi-tool chat:** Let the chatbot use `bash` to answer questions about the system. This requires breaking out of the single-cycle loop for bash calls.

5.7 07: Reaper

5.7.1 Listing

```
You are a supervisor that spawns a slow worker, then kills it via the
reaper after a timeout.

Messages arrive between cycles as ${_msg} from ${_msg_source}.
All messages are JSON objects. The msg field in send ops must be a JSON object.
This is the first cycle (${_msg} is "init").

@@worker
You are a slow worker process. Messages arrive between cycles as ${_msg}.

On every cycle, send {"text": "worker: still thinking..."} to ${_parent}
and return frames: ["@worker"]. Never terminate on your own — your parent
will kill you when it decides you've taken too long.
@@end

@@wait-for-ack
A message arrived as ${_msg} from ${_msg_source}. Check it now:
- If ${_msg} starts with "worker:": the worker is still running. This is
  your timeout signal — it has taken too long. Send {"target": "${child}"}
  to 'reaper' to kill it. Return frames: ["@wait-for-death"].
- If ${_msg} starts with "child_died:": the worker was already killed.
  Go to @report.
Otherwise, return frames: ["@wait-for-ack"] to keep waiting.
@@end

@@wait-for-death
The reaper was asked to kill the worker. A message arrived as ${_msg}.
- If ${_msg} starts with "child_died:": the worker is dead. Go to @report.
- Otherwise: keep waiting. Return frames: ["@wait-for-death"].
@@end

@@report
The worker was killed by the reaper. Send {"text": "Supervisor: worker timed out,
killed via reaper. Death notice: ${_msg}"} to 'human'. Then terminate with
an empty frames array [].
@@end

Step 1:
1. Send {"text": "Supervisor: spawning slow worker..."} to 'human'.
2. Spawn a child with frames: ["@worker"], dest "child",
   "idle": true.
3. Return frames: ["@wait-for-ack"]
```

5.7.2 What to Learn

- The reaper service: send a mailbox ID to `"reaper"` to kill a descendant.
- Ancestry check: only ancestors can kill descendants (no peer-to-peer kills).
- The kill → death-notification flow: reaper kills the child, the monitor sends `child_died:` to the parent.
- Multi-phase parent state machine: spawn → wait-for-ack → wait-for-death → report.
- Child spawned with `idle: true`—a daemon that restores its frame on empty stack and wakes on real messages.

5.7.3 Questions for Reflection

1. Why is there a `@wait-for-death` phase between sending to reaper and reporting? Could you skip it?
2. What if the child sends its status message *after* the parent sends to the reaper but *before* the child actually dies? Is there a race condition?
3. What does `idle: true` contribute to the worker's lifecycle?

5.7.4 Extension Projects

- **Graceful shutdown:** Instead of killing immediately, send the child a "please stop" message first. Only reap if it doesn't stop within N cycles.
- **Timeout via watchdog:** Instead of using the first worker message as the timeout signal, set a watchdog timer and reap when it ticks.
- **Pool of workers:** Spawn 3 workers, kill whichever finishes last.

5.8 08: Lucky Number — Historical Removed Pattern

The old guide used this slot for a grind + `receive` dice roller. That program is intentionally no longer reproduced here because both grind mode and the `receive` op were removed from the runtime.

5.8.1 What to Learn

- Why the old runtime surface was confusing enough to remove.
- Why current Gizmo requires explicit wakeups (`${_self}` or watchdog) instead of hidden hot-loop progress.
- Why test 09 is now the canonical dice example.

5.8.2 Questions for Reflection

1. Which parts of the old pattern were actual agent logic, and which parts were workarounds for the removed execution model?
2. Does removing grind make the language easier to prompt correctly, even if some autonomous loops take more cycles?

5.8.3 Extension Projects

- Rebuild the same autonomous roller using only message-driven wakeups.
- Compare a self-message loop against a watchdog-driven loop.

5.9 09: Lucky Number — Idle + Trap

5.9.1 Listing

```
You are a supervisor for a number-guessing game. Spawn an idle child
that rolls dice on a watchdog timer. If it rolls 1, kill it via reaper.
Use a trap for child death notification.

Messages arrive as ${_msg} from ${_msg_source}. All messages are JSON
objects. The msg field in send ops must be a JSON object. This is the
first cycle (${_msg} is "init").

@@roller
Roller child. Messages arrive as ${_msg} from ${_msg_source}.

If ${_msg} is "init":
  1. Send {"action": "every", "ms": 2000} to 'watchdog'.
  2. Return frames: [].

If ${_msg_source} is "watchdog":
  1. Send {"command": "printf \"%d\" $(shuf -i 1-6 -n 1)"} to 'bash'.
  2. Return frames: ["@1"].

Otherwise: return frames: [].
@@end

@@wait-roll
Waiting for bash result. Message: ${_msg} from ${_msg_source}.

If ${_msg_source} is "bash":
  1. Send {"text": "rolled:${_msg}"} to ${_parent}.
  2. Send {"text": "Child: I rolled ${_msg}"} to 'human'.
  3. Return frames: [].

Otherwise: return frames: ["@0"].
@@end

@@check-roll
Message: ${_msg} from ${_msg_source}.

If ${_msg} starts with "rolled:":
  Send {"text": "Parent: child ${_msg}"} to 'human'.
  Extract the number after "rolled:".
  If it is "1": send {"target": "${child}"} to 'reaper'.
  Return frames: ["@check-roll"].
```

```
Otherwise: return frames: ["@check-roll"].
@@end

@@death-handler
Child died: ${_interrupt}.
Send {"text": "Parent: child was reaped! ${_interrupt}"} to 'human'.
Return frames: [].
@@end

1. Send {"text": "Parent: spawning roller child..."} to 'human'.
2. Spawn a child with frames: ["@roller", "@wait-roll"],
   dest "child", "idle": true.
3. Register a trap: pattern "^child_died:", frames ["@death-handler"].
4. Return frames: ["@check-roll"].
```

5.9.2 What to Learn

The current supported autonomous-roll pattern:

- Child in message-driven mode with `idle: true` and a watchdog timer.
- Watchdog ticks trigger rolls; child goes idle between ticks.
- Parent uses `trap` for death handling instead of checking in every section.
- Multi-frame spawn: `["@roller", "@wait-roll"]` gives the child a two-frame stack.

Structural summary:

Aspect	**09 (Idle + Trap)**	**Why it matters**
Child loop	message-driven, `${_msg}`	One semantics everywhere in the runtime
Child pacing	watchdog-driven	Real mailbox wakeups; no hidden progress
Death handling	trap fires anywhere	No need to special-case `child_died:` in every frame
Cycles/roll	more than the removed hot-loop pattern	Simpler semantics were chosen over shaving cycles

5.9.3 Questions for Reflection

1. The child is spawned with `["@roller", "@wait-roll"]`—two frames. What happens to `@wait-roll` when the child returns `frames: []` after setup?
2. What happens if a watchdog tick arrives while the child is waiting for bash? Does the tick get lost?

5.9.4 Extension Projects

- **Adaptive timing:** Start the watchdog at 2000ms. After each roll, halve the interval. How fast can you go before ticks start racing the LLM?
- **One-shot timer:** Replace `{"action": "every", "ms": 2000}` with `{"action": "after", "ms": 2000}` and have the child request a new timer after each roll. Compare behavior.
- **Self-message variant:** Replace watchdog ticks with explicit `send` to `${_self}` after each roll.

5.10 10: Marketplace

This example previously used grind-mode peers that blocked internally with `receive`. That version is now historical and has been removed from the guide.

5.10.1 What to Learn

The current lessons still stand:

- **Disowned peers:** `disown: true` creates fully independent agents with no `${_parent}`.
- **Blackboard as service directory:** each agent writes its address, others read it to discover peers.
- **Binding-conditional state machines:** bank and store each run a multi-phase program in a single frame with `@0` loops and binding checks.
- **Cross-agent coordination:** three agents (coordinator, bank, store) coordinate through messages and shared state without any hierarchical relationship.
- **Retry logic:** the store re-reads `bank_mb` if the bank hasn't registered yet.

5.10.2 Questions for Reflection

1. Why is `disown: true` used? What would change if the coordinator had a parent-child relationship with the bank and store?
2. The store checks `if ${bank} starts with "agent_"` to detect whether the bank has registered. Why? What value does `${bank}` have if the bank hasn't written to the blackboard yet?
3. There's a race condition: the store may try to read `bank_mb` before the bank writes it. How does the binding-conditional pattern handle this?
4. The `@wait-result` section has strong anti-re-execution language ("Do NOT spawn any agents"). Why?

5.10.3 Extension Projects

- **Multiple stores:** Spawn 3 store agents that all query the bank. The bank should handle all 3 requests.
- **Bank with state:** Give the bank a running balance. Each request deducts from it. Use the blackboard to persist state.
- **Named agents:** Use `"name": "bank"` and `"name": "store"` instead of blackboard discovery. How does this simplify the code?

5.11 11a: Named Spawn

5.11.1 Listing

```
You are a supervisor that spawns a named child worker.

Messages arrive between cycles as ${_msg} from ${_msg_source}.
All messages are JSON objects. The msg field in send ops must be a JSON object.

@@worker
You are a named worker. Your mailbox ID should be "myworker".
Send {"text": "hello from ${_self}"} to 'human', then terminate with empty frames
[].
@@end

This is the first cycle (${_msg} is "init"). Do all of these steps:
1. Send {"text": "Spawning named worker..."} to 'human'.
2. Spawn a child with frames: ["@worker"], dest "kid",
   "name": "myworker".
3. Return frames: []. Terminate immediately after spawning.
```

5.11.2 What to Learn

- The `name` field on `spawn` gives a child a custom mailbox ID.
- `${_self}` in the child reflects the custom name (`"myworker"`).
- Parent terminates immediately after spawning—doesn't wait for the child. The child runs independently.
- Named agents are easier to address in logs, traces, and inter-agent communication.

5.11.3 Questions for Reflection

1. What happens if you run this test twice in the same runtime (i.e., two agents both try to spawn a child named `"myworker"`)?
2. The parent terminates before the child runs. Does the child still execute? Why?
3. What's the difference between `"name": "myworker"` on spawn and `--name myworker` on the CLI?

5.11.4 Extension Projects

- **Named communication:** Spawn a named child, then have a *second* child send a message to it by name (not by a binding from spawn).
- **Name collision:** Intentionally spawn two children with the same name. Observe the op error recovery behavior (`_op_error` binding).

5.12 11b: Each Hello

5.12.1 Listing

```
You are a one-shot greeter for the --each test. Send {"text": "Hello from
${_self}!"}
to 'human', then terminate with empty frames [].
```

5.12.2 What to Learn

- The `--each` CLI flag spawns one agent per positional file.
- Each agent is fully independent with its own mailbox ID, bindings, and lifecycle.
- `${_self}` differs between agents, proving they are separate processes.

Run with:

```
elixir gizmo.exs --each test/11b_each_hello.txt test/11b_each_hello.txt
```

5.12.3 Questions for Reflection

1. Both agents use the same boot frame file. How do they end up with different `${_self}` values?
2. Can `--each` agents communicate with each other? How would you set that up?
3. What's the difference between `--each a.txt b.txt` and spawning two children from a parent agent?

5.12.4 Extension Projects

- **Each with boot:** Run `--each --boot sys.txt a.txt b.txt`. Write `sys.txt` as a shared boot frame that provides common sections, and `a.txt`/`b.txt` as different tasks.
- **Cross-agent comms:** Use `--each` with `--name` on two agents. Have them discover each other via the blackboard and exchange messages. (Note: `--each` and `--name` can't be combined directly —use named spawn or blackboard discovery instead.)

6 Appendix B: Dead Ends

These are approaches tried during Gizmo's development that didn't work. Understanding *why* they failed is as instructive as understanding the patterns that succeeded.

6.1 Positional Args Stack (`$1`, `$2`, ...)

The original design pushed `receive` and `fork` results onto a positional stack. `$1` was the most recent, `$2` the one before that. Indices shifted on every push, the LLM had to count backwards to track provenance, and the stack grew without bound.

Replaced by: Named bindings via `dest` on `spawn` plus runtime wake bindings like `${_msg}`. Named, stable, self-documenting.

6.2 Text-Based `<ops>`/`<frames>` Parsing

The original design had the LLM emit ops in XML-ish delimited text blocks. Parsing was fragile and ambiguous.

Replaced by: The `eval_response` forced tool call. The LLM returns typed JSON; the provider enforces the schema. No parser needed.

6.3 `spawn_link` for Children

Children were `spawn_link`ed to parents. A child crash killed the parent, cascading up the tree.

Replaced by: `spawn` (no link) + `Process.monitor`. Child death becomes a `child_died:` *message*, not a contagion.

6.4 `fork`/`join` as Primitives

The original op set was `send`, `receive`, `fork`, `join`. `join(msg)` sent a message to the parent and terminated. But `join` is just `send(parent, msg)` + `frames: []`. And `fork`'s frame-popping `n` parameter solved a problem that doesn't exist (the LLM already controls what frames it returns).

Replaced by: `spawn` + `${_self}`/`${_parent}` bindings. Termination is just `frames: []`. Send first if you have a result to deliver.

6.5 Grinding Eval Loop as Default

The eval loop originally hot-looped: call LLM, execute ops, loop immediately. A one-shot agent burned 50 LLM calls spinning on an idle boot frame before the cycle limit killed it.

Replaced by: Message-driven eval loop. Agents sleep between cycles and wake on messages. Autonomous repetition now requires an explicit self-message or watchdog tick.

6.6 Idle-by-Default

When frames drained to `[]`, the runtime restored the boot frame and idled. One-shot agents couldn't terminate without hitting the cycle limit.

Replaced by: Terminate-on-exhaust as default. `--idle` is opt-in for daemon-style agents.

6.7 `untrap` as a Separate Op

`trap(pattern, [])` and `untrap()` were semantically identical. Two ops for one concept.

Replaced by: `trap(pattern, [])` with empty frames clears the trap.

6.8 Deferred Frame Transitions

Returning `frames: ["@quit"]` in message-driven mode blocks on the inter-cycle wait. The quit frame never executes because no message arrives.

Fix: Inline terminal behavior in the current cycle. Don't defer to a frame that will block.

6.9 Section-Based State Machines in Children

Children defined as `@@sections` in the parent tried to use `["@step2"]` to transition phases. Children don't inherit parent sections—the reference stays literal.

Fix: Single-frame or section-based state machines driven by `${_msg}` and explicit continuation frames.

6.10 Checking `${_msg}` in Grind-Mode Children

Grind-mode children expected `${_msg}` to update between cycles. It doesn't—`_msg` is only bound by the inter-cycle message wait, which grind mode skips.

Removed entirely: the runtime no longer has grind mode, so `${_msg}` now follows one semantics everywhere.

7 Appendix C: How Gizmo Compares

Gizmo occupies an unusual point in the design space. This appendix compares it to related systems —from the simplest (a plain LLM chat) to the most formal (the π-calculus)—to clarify what Gizmo is and is not.

7.1 vs. Stock LLMs (ChatGPT, Claude, etc.)

A stock LLM is a stateless text-to-text function. You send a prompt, you get a response. There is no execution loop, no persistent state between calls, and no mechanism for the model to take actions in the world. Tool use (function calling) exists, but is orchestrated by *your* code—the model emits a structured request, your application dispatches it, and you feed the result back. The model has no agency over this process; it is a passive oracle.

Gizmo inverts this. The LLM is inside a loop that *it controls*. Each cycle, the model reads its context stack, emits ops to execute and frames to continue with. It decides what to do, what tools to call, when to spawn children, and when to stop. The runtime is the executor; the LLM is the program counter. A stock LLM is a function you call. A Gizmo agent is a process that runs.

7.2 vs. OpenClaw

OpenClaw is a batteries-included AI agent framework: gateway routing across chat platforms (WhatsApp, Telegram, Slack), a skill marketplace, cron scheduling, and a memory system backed by Markdown files with hybrid vector/BM25 search. It runs a ReAct loop internally—the LLM reasons, selects a tool, observes the result, and loops.

The key architectural differences:

- **Scope.** OpenClaw is a full-featured personal assistant with 100+ skills and multi-channel integration. Gizmo is a minimal runtime with three ops.
- **Agent composition.** OpenClaw runs a single agent with modular skills. Gizmo supports hierarchical and peer-to-peer multi-agent coordination via `spawn`, `send`, and mailbox wakeups.
- **Control flow.** OpenClaw's ReAct loop is sequential—one thought, one action, one observation. Gizmo agents run concurrently on the BEAM, communicating via mailboxes.
- **State.** OpenClaw externalizes state to Markdown files on disk. Gizmo keeps state in per-process bindings and the context stack, with the blackboard as optional shared memory.
- **Memory.** OpenClaw has a sophisticated memory system with temporal decay and deduplication. Gizmo has no built-in memory beyond bindings and the blackboard.

OpenClaw optimizes for breadth of capability. Gizmo optimizes for composability from minimal primitives.

7.3 vs. ReAct

ReAct (Reason+Act, Yao et al. 2022) is the dominant pattern for LLM tool use. The agent alternates between three phases: *thought* (free-text reasoning), *action* (structured tool call), and *observation* (tool result appended to context). The loop repeats until the LLM emits a terminal action.

Gizmo differs structurally:

- **Concurrency.** ReAct is single-threaded. One agent, one loop, one tool call at a time. Gizmo agents are concurrent BEAM processes that communicate via message passing. A parent can spawn children that run in parallel.
- **State representation.** ReAct keeps all state in the context window—the growing concatenation of thoughts, actions, and observations. Gizmo separates concerns: bindings for named values, the context stack for the prompt, and messages for inter-process communication.
- **Control flow.** ReAct's control flow is emergent from token prediction. The LLM "decides" what to do by generating text that a harness parses. Gizmo's control flow is explicit: the LLM returns structured JSON with typed ops, and the runtime dispatches them deterministically.
- **Composition.** ReAct has no native mechanism for agent-to-agent communication. Multi-agent setups require external orchestration. Gizmo's `spawn`/`send` plus mailbox-driven wakeups make multi-agent coordination a first-class concern.
- **Error handling.** ReAct hopes the LLM notices when something goes wrong. Gizmo has supervision trees, death monitors, the `trap` op for structured failure handling, and op error recovery (`_op_error`/`_pending_ops` bindings) for LLM-generated bad ops.

ReAct is simple to implement and reason about for single-agent tasks. Gizmo is what you reach for when agents need to coordinate, fail gracefully, or run in parallel.

7.4 vs. LangGraph

LangGraph models agent workflows as directed graphs with typed state. Nodes are Python functions (or LLM calls); edges define transitions. Conditional edges let developer-written router functions inspect state and choose the next node. Execution follows a Pregel-style superstep model with synchronization barriers.

The fundamental contrast is *who is in the driver's seat*:

- **Developer vs. LLM control.** In LangGraph, the developer defines the graph topology in code. The LLM fills in content at specific nodes and may influence conditional edges, but only through developer-written gate functions. In Gizmo, the LLM defines its own continuation—it returns frames that *become* the next prompt. The "graph" is implicit and dynamic, shaped by the model's output each cycle.
- **State model.** LangGraph uses a shared typed dictionary with reducer functions for merging concurrent updates. Gizmo uses per-process bindings with message passing between processes —no shared mutable state.
- **Concurrency model.** LangGraph runs parallel nodes within a superstep, then synchronizes. Gizmo processes are fully asynchronous—no synchronization barriers, no supersteps. Coordination happens through messages.
- **Persistence.** LangGraph checkpoints state at every superstep, enabling time-travel debugging and fault recovery. Gizmo has no built-in checkpointing; process state lives only in memory.

LangGraph gives the developer fine-grained, verifiable control over agent workflows. Gizmo gives the LLM autonomy within the constraints of a minimal op set and prompt design.

7.5 vs. Erlang/OTP

Gizmo is explicitly modeled on Erlang/OTP, and the resemblance is deliberate:

Concept	**Erlang/OTP**	**Gizmo**
Process	Erlang process (lightweight, preemptive)	Agent process (`:proc_lib` wrapper)
Mailbox	Per-process message queue	Mailbox Registry (process mailbox for delivery)
Supervision	`Supervisor` / `DynamicSupervisor`	`Gizmo.Supervision` / `Gizmo.AgentSupervisor`
Failure	`link` / `monitor` / `EXIT` signals	`Process.monitor` + `child_died:` messages
State	Tail-recursive loop with accumulator	Context stack + bindings, rewritten each cycle
Program	Deterministic Erlang code	Non-deterministic LLM output

The critical difference is the last row. An Erlang process executes deterministic code that the developer wrote. A Gizmo agent executes whatever the LLM returns—which is non-deterministic, may hallucinate, and requires prompt engineering to steer. Gizmo borrows Erlang's *execution model* (processes, mailboxes, supervision) but replaces its *computation model* (pattern matching on messages, tail recursion) with LLM inference over a context stack.

This means Gizmo inherits Erlang's strengths (fault tolerance, concurrency, message-driven architecture) but *not* its guarantees (determinism, exhaustive pattern matching, formally verifiable behavior). An Erlang process that handles message `X` will always do the same thing. A Gizmo agent that receives message `X` will do whatever the LLM decides, given the current prompt.

7.6 vs. the π-Calculus

The π-calculus (Milner, 1992) is a formal model of concurrent computation built on three primitives: *send* a name on a channel, *receive* a name from a channel, and *create* a new channel. Processes run in parallel and synchronize through channels. Channel names can be sent as messages, enabling dynamic reconfiguration of communication topology—a process can receive a channel name it didn't know about and start communicating on it.

Gizmo's op set maps onto the π-calculus surprisingly directly:

π-calculus	**Gizmo**	**Notes**
$\overline{x}\langle\ y\ \rangle$ (send y on x)	`send(mailbox, msg)`	Fire-and-forget to a named mailbox
$x(y)$ (receive y from x)	`${_msg}` on the next wakeup	Mailbox-driven wakeup rather than an explicit receive op
$\nu\ x$ (create channel x)	`spawn(dest: "x")`	New process = new mailbox ID
$P \mid Q$ (parallel composition)	`spawn` creates concurrent processes	Agents run on the BEAM
$!P$ (replication)	`@0` loop / idle restore	Persistent behavior

The key difference is that in the π-calculus, every process is a precisely specified term in a formal algebra. You can prove properties about it: deadlock freedom, bisimulation equivalence, type safety (via session types). A Gizmo agent's "program" is a natural-language boot frame interpreted by an LLM. You cannot prove anything about it. The same boot frame can produce different behavior on different runs, or with different models, or even on the same model with different temperatures.

Gizmo is the π-calculus with the computation model replaced by vibes.

This is not entirely a joke. The structural correspondence means that techniques from the process calculus literature—session types for communication protocols, bisimulation for behavioral equivalence, type-and-effect systems for resource tracking—are *conceptually applicable* to Gizmo, even if they can't be formally verified. The pledges described in Section 4 are an informal version of session types: constraining who an agent can talk to and what it can say.

7.7 vs. Gas Town

Gas Town (Steve Yegge, 2026) is a multi-agent workspace orchestrator written in Go. It coordinates 20–30 Claude Code or Codex instances working in parallel across one or more codebases. The name is a Mad Max reference; the operational metaphor is a factory floor.

Gas Town is not a runtime for defining agent behavior—it is a *management layer* on top of existing coding agents. Its key abstractions:

- **Beads:** atomic units of work stored as JSONL in Git. A persistent issue tracker that survives agent session crashes.
- **Hooks:** Git worktree-based queues. Work is "slung" onto an agent's hook; the agent picks it up via the GUPP protocol ("Git Up, Pull, Push").
- **Hierarchical roles:** Mayor (human-facing coordinator), Witness (supervises workers), Polecats (ephemeral grunt workers), Refinery (merge queue manager). Clear chain of command prevents agents from interfering with each other.
- **Sessions are disposable:** when context runs out, new sessions spawn with identity and work restored from Git. Agent memory lives in Beads, not the context window.

The architectural contrast with Gizmo:

- **Abstraction level.** Gas Town orchestrates full coding agent sessions (Claude Code). Gizmo *is* the agent runtime—it defines how an agent thinks, acts, and communicates from four primitives.
- **State location.** Gas Town externalizes all state to Git (beads, hooks, worktrees). Gizmo keeps state in-process (bindings, context stack, message queues). Both agree that LLM context windows are ephemeral, but solve it differently.
- **Coordination mechanism.** Gas Town uses git-based handoffs and role hierarchies. Gizmo uses message passing between concurrent BEAM processes. Gas Town's GUPP protocol is deterministic file-system polling; Gizmo's `send`/`receive` is async message delivery.
- **Agent identity.** Gas Town agents carry persistent identity via Role Beads that survive restarts. Gizmo agents are `:temporary` processes—if they die, they're gone (though `child_died:` notifies the parent).
- **Scale target.** Gas Town is designed for 10–30 parallel agents burning $100/hour in API costs. Gizmo targets small process trees where the interesting question is *how agents compose*, not how many you can run.

Gas Town and Gizmo share the Erlang-inspired insight that agent systems need supervision hierarchies and message-based coordination. They diverge on where to draw the boundary: Gas Town wraps existing agents in an orchestration layer; Gizmo builds the agent from scratch.

7.8 vs. Ralph Loops

The Ralph Loop (Geoffrey Huntley, 2025) is the simplest possible agent architecture: an outer `while` loop that repeatedly invokes an LLM coding agent against the same task specification until external verification confirms completion. Named after Ralph Wiggum from The Simpsons—cheerful, persistent iteration despite repeated setbacks.

In its most distilled form:

```
while :; do
  cat PROMPT.md | coding_agent
done
```

Each iteration spawns a *fresh* agent session with a clean context window. Progress is tracked on the filesystem (committed code, progress files, test results), never in the LLM's context. When the agent tries to exit, a stop hook checks whether the task is actually done. If not, a new iteration begins.

The pattern's power is in what it *doesn't* do:

- No process model. One agent, one repo, one task.
- No message passing. No concurrency. No coordination.
- No in-context state. Disk is the only memory.
- No agent self-assessment for termination. External verification decides.

Compared to Gizmo:

- **The Ralph Loop overlaps with one Gizmo idiom.** A self-renewing agent that keeps reusing one frame is structurally similar: the runtime re-invokes the LLM each cycle, and the frame persists. The difference is that Gizmo keeps bindings and messages in-process, while the Ralph Loop offloads everything to the filesystem.
- **Context freshness.** Ralph Loops get a clean context window each iteration, eliminating "context rot." Gizmo agents accumulate bindings and the context stack grows and shrinks across cycles—which is powerful for multi-step tasks but can degrade over long runs.
- **Composition.** The Ralph Loop is deliberately non-compositional. It solves one task at a time. Gizmo's `spawn`/`send` plus mailbox wakeups exist precisely because interesting problems require multiple agents coordinating. The Ralph Loop's answer to coordination is "don't."
- **Termination.** Ralph Loops use external verification (test suites, linters, string matching). Gizmo agents self-terminate via `frames: []`. Neither trusts the LLM's self-assessment, but they enforce it differently: external script vs. runtime semantics.

The Ralph Loop is what you use when the problem is "keep hammering until the tests pass." Gizmo is what you use when the problem is "coordinate multiple agents that need to talk to each other."

8 Appendix D: Developer Cheatsheet

8.1 Eval Cycle

Wake → bind `_msg`/`_msg_source` → build prompt (preamble + boot + frames) → LLM → interpolate ops & frames → execute ops → replace frames → sleep until the next mailbox message.

Interpolation happens BEFORE ops execute. If an op triggers a future response, that response is seen on the next cycle as `${_msg}`.

8.2 Ops

Op	Syntax	Blk?	Binds	Notes
`send`	`{"op":"send", "mailbox":"id", "msg":{...}}`	No	—	Fire-and-forget. `msg` is a JSON object. Both fields interpolated.
`spawn`	`{"op":"spawn", "frames":[...], "dest":"name", ...}`	No	`name`	Child starts with `_msg="init"`. Options: `idle` (bool), `disown` (bool, false), `name`/`model` (string, inherit).
`trap`	`{"op":"trap", "pattern":"regex", "frames":[...]}`	No	—	Interrupt handler. Empty `frames` clears. Sets `_interrupt`/`_interrupt_source`.

8.2.1 Interpolation

Syntax	Expansion
`${name}`	Binding value (literal if unbound)
`@N`	Frame at index N
`@name`	Named section `@@name...@@end`
`$$` / `@@`	Literal `$` / `@`

Sections: `@@name…@@end` in boot frame. Non-greedy (first `@@end`). Keep flat.

8.2.2 Runtime Bindings

Name	Set By	Description
`_self`	always	This agent's mailbox ID
`_parent`	if spawned	Parent's mailbox ID
`_msg`	each cycle	Text summary of wake message. `"init"` on cycle 0.
`_msg_source`	each cycle	Sender ID. `"runtime"` on cycle 0.
`_payload`	each cycle	Full JSON of wake message. `"{}"` on cycle 0.
`_interrupt`	trap fire	Matched message content
`_interrupt_source`	trap fire	Matched message sender

8.3 Well-Known Services

Mailbox	Send	Response	Notes
`human`	`{"text": "..."}`	*(none)*	Print to stdout
`human_input`	`{"prompt": "..."}`	`{"text": line}`	Print prompt, read stdin line
`bash`	`{"command": "..."}`	`{"text": out, ...}`	Async shell. `--bash-timeout` applies.
`blackboard`	`{"action": "write/read", ...}`	`{"text": "ok/val", ...}`	Shared key-value store
`exception`	*(internal)*	*(none)*	Logs agent errors to stderr
`reaper`	`{"target": "mb_id"}`	*(none)*	Force-kill descendant. Ancestor-only.
`watchdog`	`{"action": "every/after/cancel", ...}`	`{"text": "tick"}`	Periodic/one-shot tick delivery
`pager`	`{"action": "open", "path": "..."}`	`{"text": "opened:sid:N lines", ...}`	Spawns session: `next`/`prev`/`search`/`goto`/`close`
`batch`	`{"requests": [...], "timeout": N}`	`{"text": "batch complete: N/M", "results": [...]}`	Fan-out parallel requests, collect all results
`eval`	`{"code": "...", "timeout": N}`	`{"text": "result", "type": "..."}`	Evaluate Elixir expression with AST sandboxing
`factory`	`{"action": "create/destroy/list", ...}`	`{"text": "ok", "name": "..."}`	Create custom stateful services. Workers get own mailbox.

8.4 CLI Flags

Flag	Effect
Files	
`<file> [file...]`	Boot frame file(s). Multiple = stacked frames.
`--boot <f>`	Separate boot frame (always in system prompt).
`--init <f>`	Write starter template and exit.
`--each`	One agent per positional file.
Modes	
`--idle`	Restore boot frame on frame exhaust (daemon).
`--watchdog <ms>`	Send `"tick"` (from `"watchdog"`) every N ms.
`--max-cycles <N>`	Cycle limit (default 50, 0 = unlimited).
`--bash-timeout <ms>`	Bash timeout (default 60000, 0 = none).
Model	
`--model <id>`	LLM model (default: env or `claude-sonnet-4-20250514`).
`--thinking`	Extended thinking (Anthropic only).
`--list-models`	List models from all backends.
`--runtime <f>` / `--dump-runtime <f>`	Custom runtime preamble / dump built-in.
Debug & Trace	
`-v` / `-vv` / `-vvv`	Increasing verbosity.
`--log-timings` / `--log-full-prompts`	Show timing / full prompt each cycle.
`--dry-run`	Print initial prompt and exit.
`--trace` / `--trace-file <f>`	NDJSON trace to stderr / file.
`--trace-service` / `--trace-messages`	Include service / message-routing events.
`--test`	Run smoke tests and exit.

8.4.1 Key Patterns

- **One-shot:** No ops, `frames: []`. Simplest agent.
- **Service call:** `send`, continuation frame, `${_msg}` next cycle.
- **`@0` loop:** `frames: ["@0"]` re-executes. Beware replaying setup.
- **Self-renewing loop:** send to `${_self}` or schedule a watchdog tick, then continue on the next wakeup.
- **Binding-conditional FSM:** "if `${x}` is in bindings" branches.
- **Named sections:** `@@step1`…`@@end`, transition via `["@step2"]`.
- **Interactive:** `send` to `human_input`, `${_msg}` next cycle, `@0`.
- **Idle daemon:** `--idle`, frames drain, boot restores, wait.

8.4.2 Pitfalls

1. `${_msg}` is not a future — it's this cycle's wake message.
2. Terse continuations (`"step2"`) lose context. Be explicit.
3. `@0` replays *everything* in the frame. Use `@@loop` sections.
4. Same-cycle `${_msg}` is stale for work you just triggered.
5. `@0` does not create progress on its own; a real wakeup message is required.
6. Children can't see parent `@@sections`. Use conditionals + `@0`.
7. Boot frame re-executes every cycle. Wrap setup in `@@section`.
8. `frames: ["@quit"]` hangs in msg-driven. Inline + `[]` instead.